



“DriverControl”

Saúl Santos Mesones

Desarrollo de Aplicaciones Multiplataforma

(Virtual)

Curso 2025/2026

* Todos los datos que aparecen en la aplicación no son reales (datos de conductores, vehículos, ubicaciones, etc.) *

ÍNDICE DE CONTENIDOS:

1. Estudio del problema y análisis del sistema	4
1.1. Introducción	4
1.2. Finalidad	4
♦ Objetivo general	4
♦ Objetivos específicos	5
1.3. Requisitos	5
1.4. Servicios que ofrecerá el sistema	7
1.5. Prevención de riesgos laborales	8
2. Recursos	8
2.1. Software	8
2.2. Hardware	9
2.3. Documental	9
3. Planificación	9
3.1. Planificación temporal (Cronograma)	9
3.2. Planificación económica (Análisis de costes)	11
3.3. Estudio de mercado, legalidad y viabilidad	13
4. Desarrollo y pruebas	16
4.1. Estructura de la base de datos	16
4.2. Estructura de la aplicación	20
4.3. Diseño de la aplicación	30
4.4. Tareas implementadas	31
4.4.1. Tarea 1: Inicio de sesión	31
4.4.2. Tarea 2: Visualización del dashboard	32
4.4.3. Tarea 3: Gestión de viajes	32
4.4.4. Tarea 4: Gestión de conductores y vehículos	33
4.4.5. Tarea 5: Registro de conducción y descansos	34
4.4.6. Tarea 6: Actualización de ubicación y compañeros cercanos	36
4.4.7. Tarea 7: Aviso de avería	36
4.4.8. Tarea 8: Generación de informes	38
4.4.9. Tarea 9: Contactar	38

4.4.10. Tarea 10: Gestión de administradores	38
4.5. Traducción de la aplicación	38
4.6. Permisos requeridos	39
4.7. Gestión de ubicaciones	40
4.8. Arquitectura de la aplicación	41
4.9. Diagramas del sistema	41
4.10. Pruebas del sistema	46
4.11. Control de versiones	49
5. Conclusiones Finales	50
5.1. Conclusiones	50
5.2. Grado de cumplimiento de los requisitos	50
5.3. Propuestas de mejora y ampliaciones futuras	52
6. Guías	53
6.1. Guía de uso	53
♦ Administrador (master)	53
♦ Conductor	55
6.2. Guía de instalación	56
6.3. Mapa navegación del sitio	60
7. Referencias	64
8. Preguntas frecuentes	66
9. Gráficos y figuras	67
10. Anexo: arquitectura, funcionamiento, localización e informes	71

1. Estudio del problema y análisis del sistema

1.1 Introducción

La gestión de una flota de transporte, incluso en empresas pequeñas, requiere un control constante de conductores, vehículos y viajes para garantizar la seguridad, el cumplimiento de la normativa y la correcta organización del trabajo. En muchos casos, esta gestión se realiza mediante llamadas telefónicas, mensajes o anotaciones manuales, lo que puede provocar errores, pérdida de información o retrasos en la toma de decisiones.

Este proyecto consiste en el desarrollo de una aplicación móvil para Android destinada a la gestión de una pequeña flota de transporte. La aplicación permitirá al administrador controlar de forma centralizada el estado de los conductores y vehículos, crear y asignar viajes, consultar ubicaciones y generar informes periódicos. Por otro lado, los conductores dispondrán de una herramienta sencilla para consultar sus viajes, registrar tiempos de conducción y descanso, actualizar su ubicación y notificar incidencias o averías. Para completar la utilidad también existe un usuario máster, el cual es único y será el encargado de gestionar a los administradores además de poder realizar las labores disponibles para estos.

La aplicación está diseñada para ser utilizada en dispositivos instalados en los propios vehículos, por lo que se prioriza una interfaz clara, rápida y fácil de usar. Además, se ha tenido en cuenta la privacidad de los conductores, permitiendo que ciertos datos, como la ubicación entre compañeros, sean opcionales. Con este proyecto se pretende ofrecer una solución práctica y realista a una situación habitual en el entorno profesional del transporte.

1.2 Finalidad

♦ Objetivo general

El objetivo general del proyecto es desarrollar una aplicación móvil para Android que permita la gestión integral de una pequeña flota de transporte, facilitando el control de conductores, vehículos y viajes, mejorando la organización, la seguridad en carretera y la comunicación entre el administrador y los conductores.

♦ Objetivos específicos

Para alcanzar el objetivo general planteado, se establecen los siguientes objetivos específicos:

- ▶ Analizar las necesidades básicas de gestión de una pequeña flota de transporte, identificando los datos y funcionalidades más relevantes para administradores y conductores.
- ▶ Diseñar una aplicación móvil con dos perfiles de usuario claramente diferenciados, cada uno con permisos y accesos adaptados a su función dentro de la flota.
- ▶ Permitir al administrador crear, editar y eliminar conductores, vehículos y viajes, así como consultar el estado y la ubicación de la flota en tiempo real.
- ▶ Facilitar a los conductores la consulta de su viaje y vehículo asignado, así como el registro de su jornada.
- ▶ Implementar un sistema de actualización de ubicación que permita al administrador supervisar la flota y a los conductores compartir su posición con otros compañeros de forma opcional.
- ▶ Incorporar un sistema de avisos que facilite la gestión de las incidencias.
- ▶ Generar informes sobre la actividad de la flota, incluyendo avisos y tiempos de conducción, con el fin de facilitar el análisis y la planificación.
- ▶ Garantizar la seguridad y la privacidad de los datos, respetando las decisiones de los conductores en cuanto al uso y compartición de su información personal.
- ▶ Documentar adecuadamente el proyecto, justificando las decisiones de diseño y desarrollo adoptadas a lo largo del proceso.

1.3 Requisitos

El proyecto es viable tanto desde el punto de vista técnico como temporal. La aplicación se desarrolla utilizando tecnologías conocidas y trabajadas durante el ciclo formativo, como el desarrollo de aplicaciones Android, el uso de bases de datos y la gestión de usuarios.

Requisitos funcionales (RF)

ID	Requisito	Prioridad
RF01	El sistema debe permitir el inicio de sesión mediante usuario y contraseña, identificando el rol (administrador, conductor o master).	Alta
RF02	El administrador debe poder crear, consultar, modificar y eliminar conductores.	Alta
RF03	El administrador debe poder crear, consultar, modificar y eliminar vehículos.	Alta
RF04	El administrador debe poder crear, consultar, modificar y eliminar viajes.	Alta
RF05	El administrador debe poder visualizar en un mapa la ubicación de todos los conductores.	Alta
RF06	El administrador debe poder generar y enviar informes.	Media
RF07	El administrador debe poder contactar con conductores mediante llamada, SMS o WhatsApp.	Media
RF08	El conductor debe poder consultar su viaje y vehículo asignado.	Alta
RF09	El conductor debe poder registrar tiempos de conducción y descanso.	Alta
RF10	El conductor debe poder compartir su ubicación de forma opcional con otros conductores.	Media
RF11	El conductor debe poder crear avisos de avería o incidencia.	Alta
RF12	El conductor debe poder contactar con su administrador.	Media
RF13	El usuario master debe poder gestionar (CRUD) a los administradores.	Alta
RF14	La aplicación debe estar traducida al inglés, portugués y rumano según idioma del dispositivo.	Baja

Tabla 1. Requisitos funcionales del sistema

Requisitos no funcionales (RNF)

ID	Requisito	Prioridad
RNF01	La aplicación debe ser compatible con Android 8.0 o superior.	Alta
RNF02	La aplicación debe requerir conexión a Internet para la sincronización de datos.	Alta
RNF03	La ubicación del conductor debe actualizarse cada 5 minutos para optimizar el consumo de batería.	Media
RNF04	La interfaz debe ser clara y con botones de tamaño adecuado para su uso en vehículos.	Alta
RNF05	Los datos deben almacenarse en Firebase	Alta
RNF06	La aplicación debe estar desarrollada en Java con Android Studio.	Alta

Tabla 2. Requisitos no funcionales del sistema

1.4 Servicios que ofrecerá el sistema

La solución planteada consiste en una aplicación móvil para Android que centraliza la gestión de una pequeña flota de transporte en una única herramienta. La aplicación ofrece funcionalidades diferenciadas según el perfil de usuario, permitiendo al administrador supervisar la flota y a los conductores registrar su actividad diaria.

La aplicación se estructura en varias pantallas principales, accesibles desde un menú claro e intuitivo. El administrador dispone de un panel principal desde el que puede gestionar los viajes, conductores, vehículos, avisos y registros de trabajo, así como crear y enviar informes, contactar con los conductores o ver en un mapa las ubicaciones de estos y de los avisos. El conductor, por su parte, accede a un dashboard simplificado donde poder consultar su viaje, datos de su vehículo, crear avisos y registrar su jornada además de ver los datos de su administrador, pudiendo contactar con él, y ver si tiene algún compañero cerca disponiendo de la posibilidad de contactarlos y de ver su ubicación. Por último, existe un usuario con rol de master que en la práctica es un administrador con la funcionalidad añadida de gestionar los administradores.

1.5 Prevención de riesgos laborales

Durante el desarrollo del proyecto se han tenido en cuenta diferentes medidas relacionadas con la prevención de riesgos laborales, especialmente aquellas asociadas al trabajo con equipos informáticos y al desarrollo de software. Además, se han considerado las recomendaciones establecidas en la Ley 31/1995 de Prevención de Riesgos Laborales y el Real Decreto 488/1997 sobre trabajos con pantallas de visualización de datos.

Riesgo	Descripción	Medidas preventivas
Ergonomía postural	Trabajo prolongado frente al ordenador (12 semanas de desarrollo)	Uso de silla ergonómica, pantalla a la altura de los ojos, descansos cada 2 horas
Fatiga visual	Exposición continua a pantallas (código, diseño, documentación)	Aplicación de la regla 20-20-20 (cada 20 min, mirar a 20 pies [6 m aprox.] durante 20 segundos), uso de filtro de luz azul
Estrés por plazos	Entregas ajustadas y carga de trabajo concentrada	Planificación temporal detallada, distribución equilibrada de tareas
Riesgo eléctrico	Uso continuo de equipos electrónicos	Instalación eléctrica en condiciones seguras, no sobrecargar regletas

Tabla 3. Riesgos laborales durante el desarrollo del proyecto

2. Recursos

2.1 Software

Para el desarrollo de la aplicación se han utilizado diversas herramientas software que facilitan tanto la programación como la gestión y documentación del proyecto. El entorno de desarrollo principal es Android Studio, utilizando el lenguaje de programación Java. Para el almacenamiento de datos se emplea una base de datos en la nube como es Firebase y como librería de mapas se ha utilizado OpenStreetMap.

2.2 Hardware

Los recursos hardware necesarios para el proyecto son mínimos. Se requiere un ordenador personal para el desarrollo de la aplicación y un dispositivo móvil Android o un emulador para realizar las pruebas de funcionamiento. No es necesario ningún hardware específico adicional.

2.3 Documental

Para la realización del proyecto se han consultado diferentes recursos documentales, como manuales de Android, documentación oficial de las herramientas utilizadas, apuntes del ciclo formativo y recursos online relacionados con el desarrollo de aplicaciones móviles y la gestión de bases de datos. Han sido de especial ayuda páginas y blogs de profesores como [abrilcode](#) y [codeandcoke](#).

3. Planificación

3.1 Planificación temporal (Cronograma)

El desarrollo del proyecto se ha planificado para un periodo aproximado de tres meses, distribuyendo las tareas en distintas fases con el objetivo de organizar el trabajo de forma progresiva y equilibrada.

Fase 1: Análisis y planificación (Semanas 1 y 2)

Duración estimada: 2 semanas

- Análisis de la idea del proyecto y definición del alcance (3 días).
- Redacción de la introducción del proyecto (2 días).
- Definición del objetivo general y de los objetivos específicos (2 días).
- Estudio de la viabilidad del proyecto (3 días).

Fase 2: Diseño de la solución (Semanas 3 y 4)

Duración estimada: 2 semanas

- Diseño de la estructura general de la aplicación (3 días).
- Diseño de la base de datos y relaciones entre entidades (4 días).
- Diseño de la navegación y de las pantallas principales (3 días).

Fase 3: Desarrollo básico de la aplicación (Semanas 5 a 7)

Duración estimada: 3 semanas

- Desarrollo del sistema de inicio de sesión y control de usuarios (5 días).
- Implementación del dashboard del administrador (5 días).
- Implementación del dashboard del conductor (4 días).
- Desarrollo de la gestión de viajes (4 días).

Fase 4: Desarrollo de funcionalidades (Semanas 8 y 9)

Duración estimada: 2 semanas

- Desarrollo de la gestión de conductores y vehículos (3 días).
- Desarrollo de la gestión de informes (3 días).
- Implementación del registro de tiempos de conducción y descansos (3 días).
- Implementación del sistema de localización de conductores (4 días).

Fase 5: Comunicación y avisos (Semana 10)

Duración estimada: 1 semana

- Implementación de los avisos (3 días).
- Integración de la comunicación mediante llamadas, sms y WhatsApp (2 días).

Fase 6: Pruebas y corrección de errores (Semana 11)

Duración estimada: 1 semana

- Realización de pruebas de funcionamiento generales (3 días).
- Corrección de errores detectados durante las pruebas (2 días).

Fase 7: Documentación y entrega final (Semana 12)

Duración estimada: 1 semana

- Elaboración de la documentación final del proyecto (4 días).
- Revisión final y preparación de la entrega (2 días).

El siguiente diagrama de Gantt muestra la planificación temporal del proyecto, distribuida en un periodo aproximado de tres meses. En él se reflejan las distintas fases del proyecto, desde el análisis inicial y el diseño, pasando por el desarrollo de las funcionalidades principales, hasta las pruebas y la documentación final.



Figura 1. Diagrama de Gantt

3.2 Planificación económica (Análisis de costes)

El desarrollo de la aplicación de gestión de flotas no supone un coste económico elevado, ya que se utilizan principalmente herramientas gratuitas o incluidas dentro del entorno educativo. Aun así, es importante realizar una estimación aproximada de los costes que tendría el proyecto en un entorno profesional, para comprender el valor de las tareas realizadas.

En cuanto al desarrollo, el coste principal corresponde al tiempo de trabajo del desarrollador. Para este proyecto se ha estimado un tiempo total de trabajo de aproximadamente 12 semanas, distribuidas entre análisis, diseño, desarrollo, pruebas y documentación. Aunque en un entorno académico este tiempo no implica un gasto directo, en un contexto profesional supondría un coste económico significativo, que en promedio podría situarse en unos 3.000-3.500 € para un desarrollador junior, considerando unas 240 horas de trabajo a una tarifa media de 12-14 € la hora.

Respecto al software, se han empleado herramientas gratuitas y accesibles para principiantes. Entre ellas se incluyen Android Studio para el desarrollo de la aplicación y Firebase para la gestión de la base de datos y servicios en la nube, ambos con planes gratuitos suficientes para proyectos pequeños. Si se necesitara ampliar el servicio o utilizar funcionalidades avanzadas de la nube, los costes serían bajos y proporcionales al tamaño del proyecto, estimándose en unos 30-50 € al trimestre.

En cuanto al hardware, únicamente se requiere un ordenador personal para el desarrollo y un dispositivo Android o un emulador para las pruebas. Estos recursos suelen estar disponibles para el alumno, por lo que no generan un coste adicional directo. En un entorno profesional, adquirir equipo nuevo podría implicar un gasto de 600–1.000 € por un portátil adecuado y 200–400 € por un dispositivo de pruebas, aunque estos equipos serían reutilizables para otros proyectos.

Por último, en un contexto profesional en España, también es necesario tener en cuenta la fiscalidad asociada al desarrollo del proyecto. En caso de realizar la actividad como trabajador autónomo, sería obligatorio darse de alta y asumir el pago de la cuota a la Seguridad Social, que puede situarse aproximadamente entre 80 y 230 € mensuales según las condiciones. Asimismo, el servicio prestado estaría sujeto al Impuesto sobre el Valor Añadido (IVA) del 21 %, que se añadiría al precio final, y a la tributación en el Impuesto sobre la Renta de las Personas Físicas (IRPF), con retenciones habituales del 15 % (o del 7 % en los primeros años de actividad). Esto implica que una parte de los ingresos estimados se destinaría al pago de impuestos y cotizaciones, reduciendo el beneficio neto del proyecto.

En conjunto, el proyecto es económicamente viable, ya que no requiere grandes inversiones y puede desarrollarse con recursos gratuitos o accesibles. Además, esta estimación permite comprender el valor real del tiempo y los recursos invertidos, proporcionando una visión clara de cómo se gestionan los costes en un entorno profesional de desarrollo de software.

3.3 Estudio de mercado, legalidad y viabilidad del proyecto

Una vez finalizado el desarrollo técnico del sistema, resulta necesario analizar el proyecto desde una perspectiva empresarial, legal y operativa, con el objetivo de evaluar su aplicabilidad en un entorno real y determinar su viabilidad como solución tecnológica dentro del sector del transporte profesional.

♦ Estudio de mercado

El sector del transporte por carretera atraviesa actualmente un proceso de modernización impulsado por la digitalización y la necesidad de mejorar la seguridad vial, la gestión de conductores y el cumplimiento de la normativa vigente. Las empresas del sector deben supervisar múltiples aspectos relacionados con la actividad diaria, como el control de incidencias, la organización del personal o la elaboración de informes administrativos.

Aunque existen soluciones tecnológicas especializadas, muchas de ellas están orientadas a grandes compañías y requieren inversiones económicas elevadas o infraestructuras complejas. Esta situación provoca que pequeñas y medianas empresas continúen utilizando métodos manuales o herramientas poco integradas que dificultan la gestión eficiente de la información.

El proyecto DriverControl se plantea como una alternativa accesible basada en tecnología móvil y servicios en la nube, permitiendo digitalizar procesos sin necesidad de grandes recursos técnicos ni económicos. La creciente adopción de dispositivos móviles en entornos profesionales favorece la implantación de aplicaciones de este tipo, lo que evidencia la existencia de un contexto de mercado favorable para soluciones ligeras, escalables y fácilmente adaptables a diferentes organizaciones.

♦ Marco legal y normativa aplicable

El sistema desarrollado gestiona información asociada a usuarios y conductores, por lo que debe respetar la legislación vigente en materia de protección de datos personales. En este sentido, el proyecto se enmarca dentro del Reglamento General de Protección de Datos (RGPD) y de la Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales, aplicables al tratamiento de información en entornos digitales.

Durante el diseño de la aplicación se han aplicado principios de seguridad orientados a limitar el acceso a la información mediante autenticación de usuarios y control de permisos según roles definidos dentro del sistema. El uso de servicios cloud proporciona

además mecanismos adicionales de protección y gestión segura de los datos almacenados.

Por otra parte, la aplicación se relaciona con el ámbito de la prevención de riesgos laborales y con la normativa del transporte por carretera, ya que permite registrar incidencias, generar informes y mejorar la trazabilidad de la actividad profesional. Aunque no sustituye a herramientas oficiales obligatorias, como el tacógrafo digital, sí constituye un sistema complementario que contribuye al control interno y a la mejora organizativa.

♦ Viabilidad

El análisis global del proyecto permite afirmar que DriverControl presenta una alta viabilidad desde los puntos de vista técnico, económico y operativo.

Desde el ámbito técnico, el sistema se apoya en tecnologías ampliamente consolidadas dentro del desarrollo móvil, como la plataforma Android, el lenguaje Java y los servicios cloud proporcionados por Firebase. Estas herramientas garantizan estabilidad, escalabilidad y facilidad de mantenimiento, además de eliminar la necesidad de infraestructuras propias de servidor, reduciendo la complejidad del despliegue y la administración del sistema.

En términos económicos, el proyecto resulta viable debido al bajo coste asociado a su desarrollo e implantación. Las herramientas utilizadas son gratuitas o disponen de planes iniciales sin coste, lo que permite implementar la solución sin realizar inversiones significativas. El modelo basado en servicios en la nube posibilita además adaptar los costes al crecimiento real del sistema, facilitando su adopción progresiva por parte de pequeñas y medianas empresas.

Desde una perspectiva operativa, la aplicación ha sido diseñada con una interfaz sencilla e intuitiva orientada a usuarios no técnicos, lo que favorece su integración en el funcionamiento habitual de una empresa sin requerir procesos complejos de formación. El uso de dispositivos móviles ya presentes en el entorno laboral permite implantar la solución de manera inmediata, reduciendo tiempos de adaptación y mejorando la eficiencia organizativa mediante la automatización de tareas administrativas.

Finalmente, el proyecto presenta un elevado potencial de evolución futura. La arquitectura implementada permite ampliar funcionalidades, integrar nuevos servicios o desarrollar módulos complementarios sin necesidad de rediseñar el sistema desde cero. Esta capacidad de crecimiento demuestra que la aplicación no se limita a un

ejercicio académico, sino que puede evolucionar hacia una solución tecnológica real aplicable en el ámbito profesional.

Para evaluar el proyecto de forma objetiva se ha realizado un análisis DAFO, identificando las debilidades, amenazas, fortalezas y oportunidades relacionadas con el desarrollo de la aplicación de gestión de flotas.

ANÁLISIS INTERNO	
FORTALEZAS	DEBILIDADES
◆ Aplicación con una utilidad práctica y real en el ámbito del transporte. ◆ Diferenciación clara de roles entre administrador y conductor. ◆ Interfaz sencilla y adaptada a dispositivos móviles instalados en vehículos. ◆ Automatización de tareas como el control de tiempos y la generación de informes. ◆ Uso de tecnologías conocidas y trabajadas durante el ciclo formativo.	◆ Funcionalidades limitadas para ajustarse al tiempo y nivel del proyecto académico. ◆ Dependencia de la conexión a internet para la actualización de datos y ubicación. ◆ Diseño enfocado a empresas pequeñas, no preparado para flotas de gran tamaño. ◆ Ausencia de funcionalidades avanzadas como mensajería interna o análisis de rutas.

Tabla 4. Análisis DAFO – Factores internos

ANÁLISIS EXTERNO	
OPORTUNIDADES	AMENAZAS

◆ Posibilidad de ampliar la aplicación con nuevas funcionalidades en el futuro. ◆ Adaptación del proyecto a otros sectores que requieran control de vehículos o personal. ◆ Integración con servicios externos como mapas avanzados o sistemas de gestión empresarial. ◆ Uso del proyecto como base para un desarrollo profesional o comercial.	◆ Existencia de aplicaciones comerciales similares ya consolidadas en el mercado. ◆ Cambios en la normativa de transporte que obliguen a modificar la aplicación. ◆ Problemas de compatibilidad con determinados dispositivos móviles. ◆ Riesgos relacionados con la gestión de datos sensibles y la privacidad de los usuarios.
--	---

Tabla 5. Análisis DAFO – Factores externos

Este análisis permite identificar los puntos fuertes del proyecto y las áreas de mejora, así como justificar las decisiones tomadas durante su desarrollo.

4. Desarrollo y pruebas

4.1 Estructura de la base de datos

La aplicación utiliza una base de datos diseñada para almacenar de forma organizada toda la información necesaria para la gestión de la flota. La estructura de la base de datos permite relacionar conductores, vehículos y viajes, así como registrar los tiempos de conducción, descansos y posibles incidencias. La base de datos del proyecto consta de las siguientes colecciones:

◆ Usuarios

La colección de usuarios almacena los datos personales básicos, como el nombre, el usuario y la contraseña, el teléfono de contacto y el estado actual. Un campo importante es el rol que permite diferenciar entre conductores, administradores y el master para que la aplicación ejecute la correspondiente funcionalidad. Además, se guarda información relacionada con la ubicación y la decisión de compartirla en el caso de los conductores o el email en los administradores.

♦ Vehículos

La colección de vehículos contiene los datos principales de cada vehículo de la flota, como la matrícula, el tipo, la capacidad y su estado actual. Cada vehículo puede estar asignado a un conductor, según la organización de la flota.

♦ Viajes

La colección de viajes registra la información relacionada con cada desplazamiento, incluyendo el origen, el destino, el conductor asignado, el vehículo utilizado, el estado del viaje y la fecha. Esta información permite consultar el historial de viajes y generar informes de actividad.

♦ Colecciones auxiliares

Por último, se incluyen colecciones auxiliares para el registro de los tiempos de conducción y descanso, así como para los avisos de avería o incidencias. Esta estructura facilita el control de la actividad diaria y permite mantener un historial limitado en el tiempo, evitando la saturación de la base de datos al disponer de una función que permite un borrado periódico.

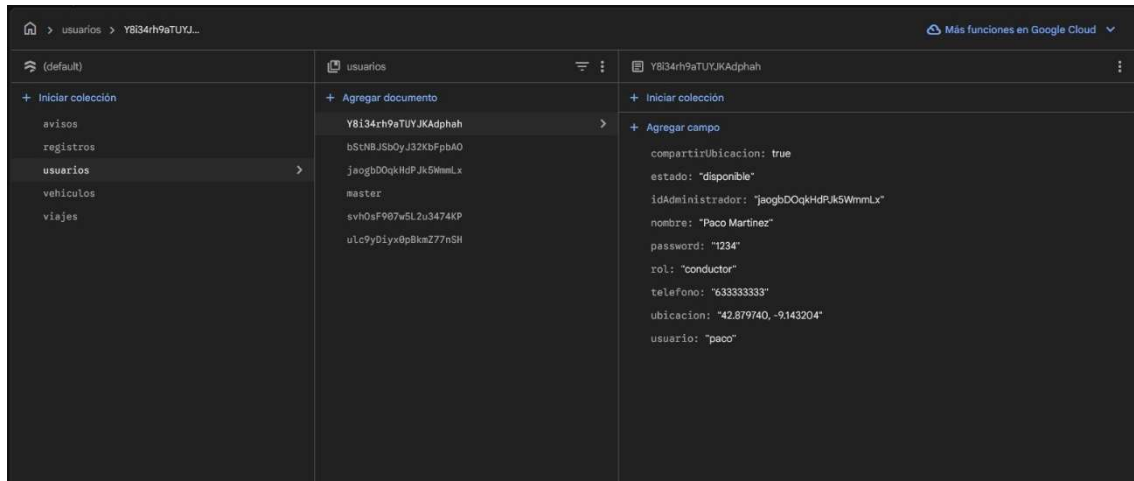


Figura 2. Estructura de la base de datos en firebase

♦ Reglas e índices

Al crear un nuevo proyecto en Firebase, las reglas de seguridad se configuran por defecto en modo de prueba, incluyendo la siguiente cláusula temporal:

allow read, write: if request.time < timestamp.date(año, mes, día);

Esta configuración, diseñada exclusivamente para entornos de desarrollo, bloquea automáticamente el acceso a la base de datos tras la fecha indicada (aproximadamente 30 días desde la creación del proyecto). Se trata de una limitación del modo de prueba, no de las reglas de seguridad en sí mismas. Firebase implementa este mecanismo para evitar que aplicaciones no configuradas correctamente queden expuestas de forma indefinida en entornos de producción.

Para que la aplicación funcione más allá de ese periodo, es imprescindible modificar manualmente dicha fecha a un valor posterior. En este proyecto se ha optado por esta solución por tratarse de un entorno académico y de pruebas. No obstante, se reconoce que esta práctica no es la recomendada por Firebase para entornos de producción reales, donde deberían implementarse reglas más complejas.

```
1 rules_version = '2';
2
3 service cloud.firestore {
4   match /databases/{database}/documents {
5
6     // This rule allows anyone with your Firestore database reference to view, edit,
7     // and delete all data in your Firestore database. It is useful for getting
8     // started, but it is configured to expire after 30 days because it
9     // leaves your app open to attackers. At that time, all client
10    // requests to your Firestore database will be denied.
11    //
12    // Make sure to write security rules for your app before that time, or else
13    // all client requests to your Firestore database will be denied until you Update
14    // your rules
15    match /{document=**} {
16      allow read, write: if request.time < timestamp.date(2026, 7, 7);
17    }
18  }
19 }
```

Figura 3. Reglas de seguridad configuradas firebase

Para el correcto funcionamiento de esta aplicación también es necesario crear índices compuestos de acceso a los datos. Firebase, a diferencia de las bases de datos relacionales, no crea índices automáticamente para consultas que incluyen ordenaciones (ORDER BY) o múltiples filtros (WHERE). Dado que la mayoría de las consultas realizadas en esta aplicación implican ordenación de resultados (por ejemplo, listar viajes por fecha, avisos por antigüedad o registros por hora de inicio), es necesario definir estos índices de forma explícita.

Los índices pueden crearse de dos maneras:

- Directamente desde Firebase Console, accediendo a la sección "Índices compuestos" y añadiéndolos manualmente.

- Mediante el aviso que genera Android Studio en el log de ejecución. Al lanzar la aplicación, si falta algún índice, Firebase muestra un error con un enlace directo para crearlo automáticamente.

ID de la colección	Campos Indexados	Alcance de la consulta	ID de índice	Estado
aviso	idConductor, fecha, __name__	Colección	CICAgO3kJK	Habilitado
registros	idConductor, fechalnicio, horainicio, __name__	Colección	CICAgJ7z4EK	Habilitado
registros	fechalnicio, horainicio, __name__	Colección	CICAgJmJEK	Habilitado
usuarios	compartirUbicacion, rol, nombre, __name__	Colección	CICAgO36pgJ	Habilitado
usuarios	rol, nombre, __name__	Colección	CICAgO39Wk	Habilitado
usuarios	idAdministrador, rol, nombre, __name__	Colección	CICAgO36pgK	Habilitado

Figura 4. Índices configurados de firebase

◆ Protección de datos y privacidad

DriverControl utiliza un modelo de base de datos independiente por cliente (un proyecto Firebase por cada empresa que utiliza la aplicación). Esta decisión se justifica por los siguientes motivos:

Motivo	Explicación
Aislamiento de datos	Los datos de un cliente no pueden ser accedidos desde la aplicación de otro cliente, garantizando la confidencialidad
Cumplimiento del RGPD	El Reglamento General de Protección de Datos (UE) 2016/679 exige que los datos personales estén protegidos y no se mezclen entre diferentes responsables de tratamiento
Control sobre la información	Cada cliente tiene control total sobre sus propios datos (conductores, vehículos, viajes, ubicaciones, etc.)
Seguridad ante brechas	Una posible vulnerabilidad en la base de datos de un cliente no afecta al resto de clientes
Portabilidad de datos	Si un cliente desea cambiar de proveedor, puede exportar únicamente sus datos sin afectar a terceros

Tabla 6. Justificación del modelo de la base de datos por cliente

Además, durante el desarrollo del proyecto se ha cumplido con la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales, garantizando que:

- Todos los datos utilizados en las pruebas de la aplicación no son reales.
- No se ha hecho uso de datos personales de terceros sin su consentimiento.
- El código fuente no contiene credenciales ni contraseñas reales.

4.2 Estructura de la aplicación

La aplicación se organiza en diferentes pantallas, accesibles según el perfil de usuario que inicia sesión. Tras el primer acceso, el sistema identifica si hay un usuario registrado y si es administrador, conductor o master y muestra la interfaz correspondiente. En caso de no existir usuario activo se lanzará la pantalla de Login. Esta comprobación es realizada mientras se muestra una pantalla de presentación y así dar tiempo a la aplicación a realizar las tareas necesarias.



Figura 5. Pantalla de inicio

♦ Login

Comenzamos por logarnos introduciendo el usuario y la contraseña tras lo cual podremos acceder a las funcionalidades que tengamos disponibles o modificar nuestras credenciales de acceso.



Figura 6. Pantalla de login

◆ Panel del administrador

El administrador accede a un panel principal o dashboard desde el que puede visualizar el estado general de la flota. Desde este panel se accede a las secciones de gestión de viajes, conductores, vehículos, avisos, registros de conducción, informes, contactos y el mapa de con las ubicaciones además de la posibilidad de cerrar sesión. Este mismo panel lo utiliza el usuario master con la funcionalidad añadida para gestionar a los administradores.

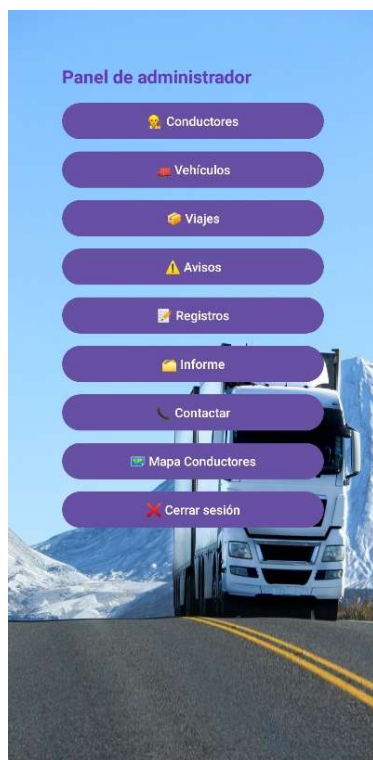


Figura 7. Panel del administrador



Figura 8. Panel del master

Gestión de conductores, vehículos, viajes, avisos y registros

En estas secciones de gestión se permite crear nuevos registros de forma rápida, asignando los datos necesarios, así como consultar detalles, editar o eliminar los existentes. La aplicación lo almacena en la base de datos para su posterior consulta. En esta parte se ha intentado realizar una estructura diferente al resto de la aplicación mediante la reutilización de la vista y que esta se adapte según el botón pulsado mostrando los campos, spinners, pickers o elementos necesarios para cada uso. El objetivo de esta estructura es dar otro enfoque al resto de la aplicación y mostrar una posible forma de reutilizar y adaptar una vista según las necesidades.

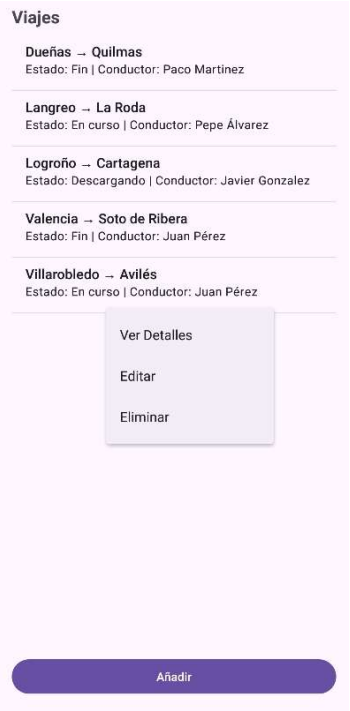


Figura 9. Lista de viajes

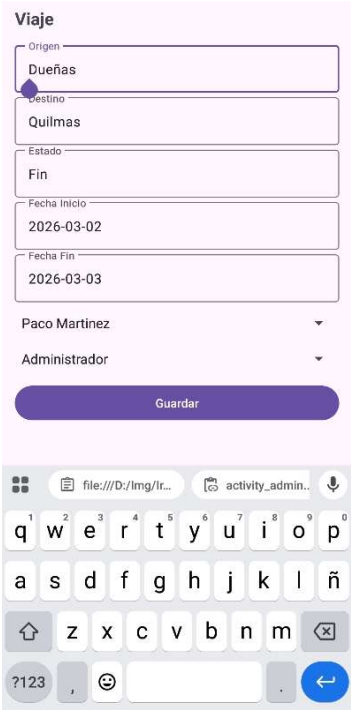


Figura 10. Editar viajes

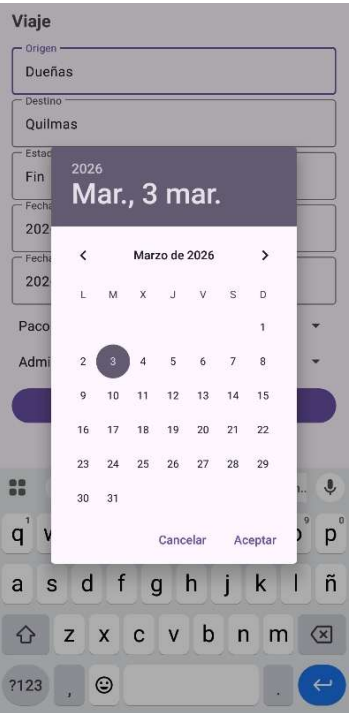


Figura 11. Picker fecha de viaje

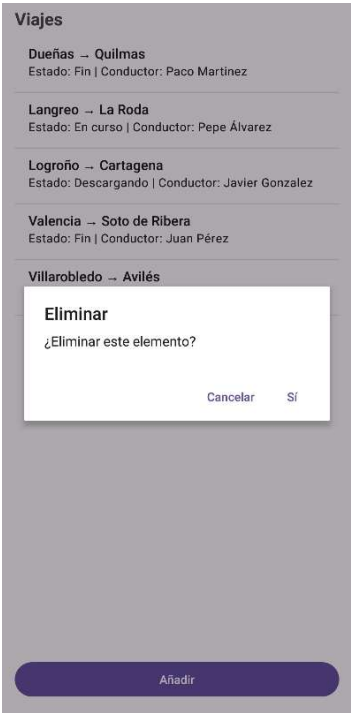


Figura 12. Confirmación eliminar

Generación de informes

Esta vista nos va generar un informe además de mostrarlo y poder enviarlo por correo electrónico en formato pdf o escrito como texto del email una vez obtenido. Para este informe el administrador podrá seleccionar un conductor concreto o que el informe sea de todos los conductores que tenga asignados. Por último, dispone de un spinner en el que seleccionar el día en el que desea programar el informe semanal de todos sus conductores, el cual será enviado a su correo desde el email del usuario master.



Figura 13. Selección conductor informe



Figura 14. Informe conductor

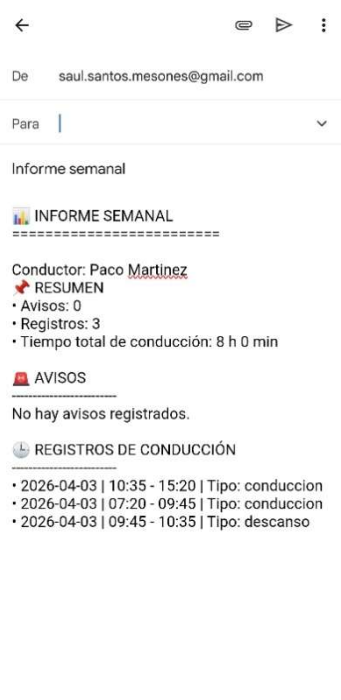


Figura 15. Informe en email

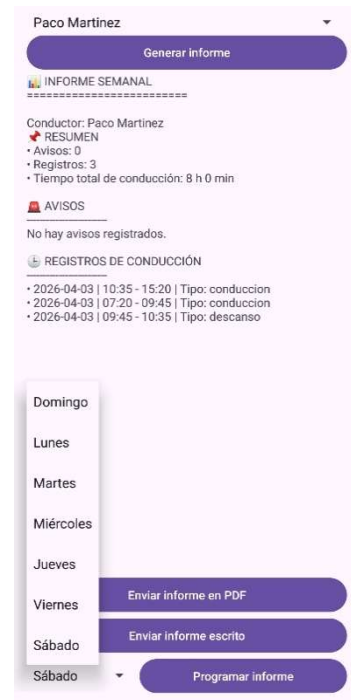


Figura 16. Selección día informe

Contactar

Si un administrador desea contactar con otro usuario dispone de una opción para ello en la cual aparecerá una ventana de diálogo con un desplegable en el que elegir con quien quiere contactar y unas opciones de contacto donde seleccionar de qué manera comunicarse entre las que puede seleccionar llamar, enviar un sms o contactar a través de WhatsApp. El Conductor dispone de la opción de contactar con el administrador que tenga asignado desde una pantalla que le muestra los datos de este. Tras pulsar el botón de contactar aparecerá una ventana emergente en la que elegir la forma de contacto.

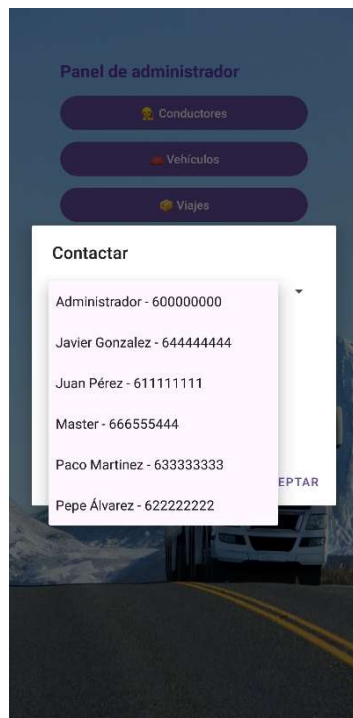


Figura 17. Selección de contacto

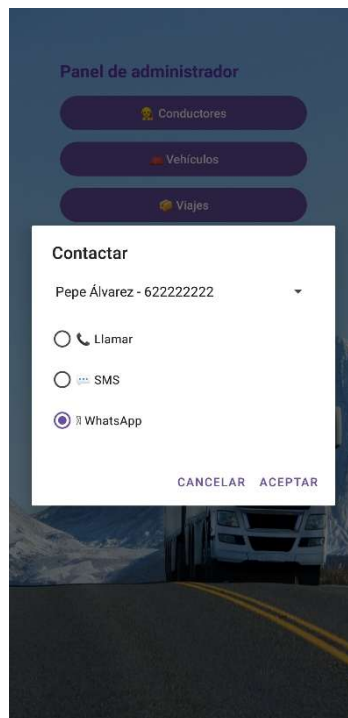


Figura 18. Ventana contacto



Figura 19. Contacto administrador

Mapa conductores

Una utilidad importante para esta aplicación es la posibilidad de ver en un mapa la ubicación de los conductores y de los avisos permitiendo tener información y tomar decisiones según las necesidades del momento. Para la rápida identificación de ambos se han utilizado diferentes indicadores en el mapa y así hacerlo más intuitivo, mostrando los conductores con el marker por defecto y uno personalizado para los avisos. En todos ellos se puede clicar y aparecerá la información correspondiente.



Figura 20. Conductor en el mapa



Figura 21. Aviso en el mapa

Gestión de administradores

Existe una opción que solo está disponible al logarse como master que es la gestión de los administradores. Esta funcionalidad se destina a la creación, consulta, modificación y eliminado de los administradores mediante una vista específica para tal fin.

Gestión de Administradores

Administrador

admin

.....

saulsantosme@yahoo.com

600000000

admin

activo

Guardar Limpiar

Administradores registrados

Administrador

Figura 22. Gestión de administradores

♦ Panel del conductor

Por su parte, el conductor dispone de un dashboard simplificado donde puede consultar el viaje asignado, la información de su vehículo, acceder al registro de tiempos de conducción y descansos, crear avisos, consultar la información del administrador que tiene asignado y contactar con él. En esta pantalla también se muestra en la parte superior la información del propio conductor y del vehículo asignado. La información del vehículo además puede verse en la pantalla específica para tal fin.



Figura 23. Panel del conductor

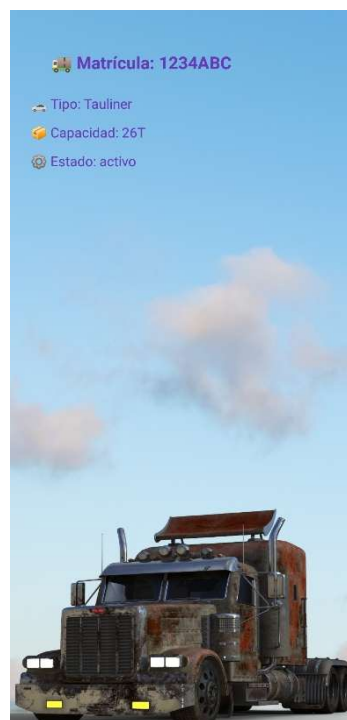


Figura 24. Datos del vehículo

Compañeros cercanos

El conductor dispone de algo que le puede ser útil en su día a día como es saber si tiene algún compañero cerca, siempre que tengan activada la opción de compartir la ubicación. El conductor podrá seleccionar el compañero del que quiere consultar donde se encuentra exactamente o contactar con él. La ubicación le aparecerá en la aplicación de maps del dispositivo y en caso de querer contactar podrá seleccionar el modo de contacto deseado estando disponible llamar, enviar sms o enviar WhatsApp.



Figura 25. Compañeros cercanos



Figura 26. Ubicación en maps

4.3 Diseño de la aplicación

El diseño de la aplicación se ha realizado teniendo en cuenta que será utilizada principalmente en dispositivos móviles instalados en los vehículos, por lo que se ha priorizado la claridad visual, la facilidad de uso y el acceso rápido a las funciones principales.

La interfaz presenta un diseño sencillo y funcional, con botones de tamaño adecuado para facilitar su uso en situaciones de movilidad. Las pantallas principales muestran únicamente la información necesaria, evitando sobrecargar al usuario con datos innecesarios.

El dashboard del administrador ofrece una serie de opciones a través de las cuales se accede rápidamente a todas las funcionalidades que incluye la aplicación para poder observar el estado de la flota mediante listados con datos claros, pudiendo acceder a más detalles en caso de desearlo. A parte de las opciones para el manejo de información, también está presente la posibilidad de contactar con otros usuarios o de ver las ubicaciones en un mapa.

En el caso del conductor, el diseño está más simplificado. El dashboard muestra sus datos, así como el vehículo actual, y los botones necesarios para interactuar con la

aplicación y poder ver o crear la información que necesita (el viaje asignado, el vehículo utilizado, registros de tiempos, ubicaciones, contacto y aviso de avería).

La navegación entre pantallas se realiza mediante menús claros e intuitivos, permitiendo al usuario volver fácilmente a la pantalla principal en cualquier momento. Este diseño garantiza una experiencia de usuario cómoda, rápida y segura.

4.4 Tareas Implementadas

4.4.1 Tarea 1: Inicio de sesión

El usuario accede a la aplicación mediante una pantalla de inicio de sesión donde introduce su nombre de usuario y contraseña. En función de las credenciales introducidas, el sistema identifica si el usuario es administrador, conductor o master y muestra la interfaz correspondiente. Esta tarea garantiza el acceso seguro a la información y la separación de permisos entre los distintos perfiles. Además, es posible modificar las credenciales de acceso mediante una ventana de diálogo donde se controla que la nueva contraseña se corresponda con un patrón preestablecido.

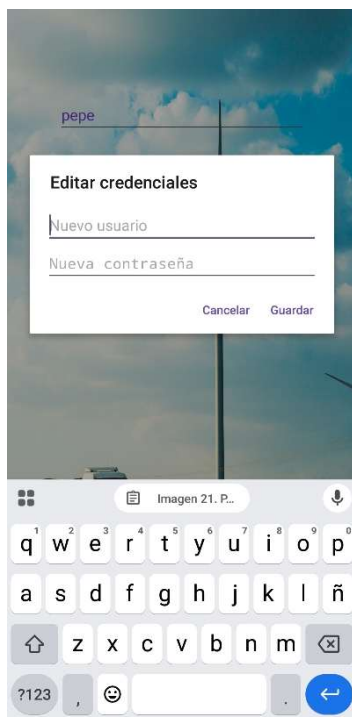


Figura 27. Editar credenciales

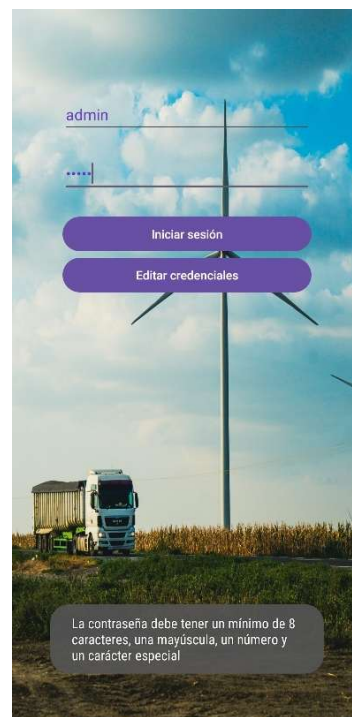


Figura 28. Contraseña no válida

4.4.2 Tarea 2: Visualización del dashboard

Tras iniciar sesión, el usuario accede a su dashboard principal. En el caso del usuario con rol de administrador o master la vista muestra las diferentes opciones, incluyendo para este último una opción a mayores que le permite la gestión de los administradores. En el caso del conductor, se muestran sus datos y los del vehículo que actualmente tiene asignado además de los botones que le van a permitir interactuar con la interfaz. Esta pantalla permite acceder rápidamente a las funciones principales de la aplicación.

4.4.3 Tarea 3: Gestión de viajes

El administrador puede crear nuevos viajes introduciendo los datos necesarios como son el origen, el destino y el conductor entre otros. Además, puede consultar el listado de viajes existentes, modificar sus datos o eliminarlos si es necesario. Esta tarea permite organizar la actividad diaria de la flota de forma sencilla y centralizada. El conductor por su parte puede ver datos relativos al viaje en una pantalla específica para tal fin.

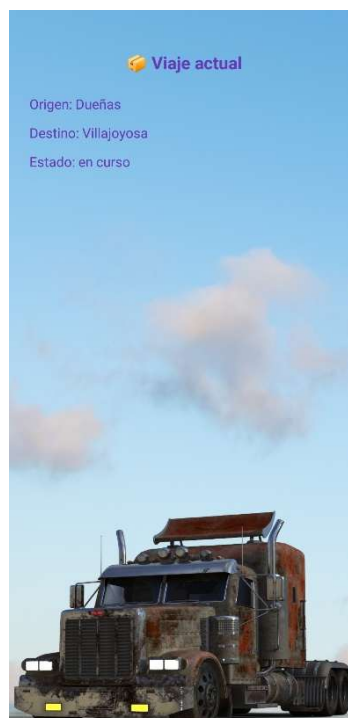


Figura 29. Viaje actual

4.4.4 Tarea 4: Gestión de conductores y vehículos

Desde las secciones correspondientes, el administrador puede dar de alta nuevos conductores y vehículos o consultar y modificar sus datos. Esta tarea es fundamental para mantener actualizada la información de la flota y asegurar una correcta asignación de recursos. Todas las listas se ordenan teniendo en cuenta sus principales datos y así facilitar su utilización (nombre, fecha o matrícula). Para introducir todos los datos se han tenido en cuenta el tipo de estos para que se introduzcan datos correctos aplicando patrones, que el teclado sea el adecuado al dato a introducir o la utilización de pickers y spinners.

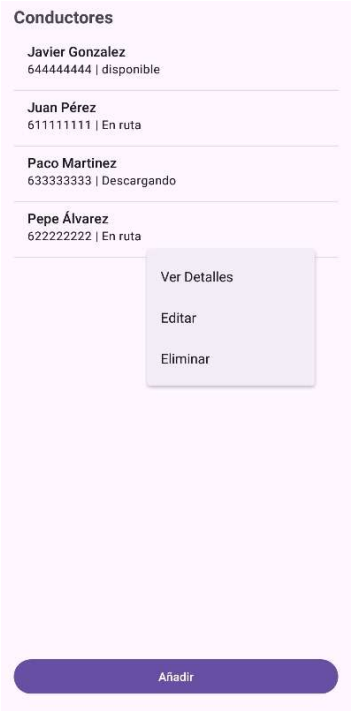


Figura 30. Lista de conductores

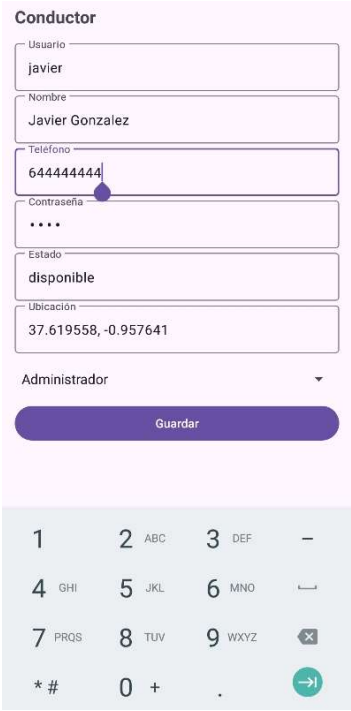


Figura 31. Editar conductor

Vehículos

1111AAA
Semitauliner | Capacidad: 25T

1234ABC
Tauliner | Capacidad: 26T

2222BBBB
Tauliner | Capacidad: 26T

5678DEF
Piso móvil | Capacidad: 24T

Añadir

Figura 32. Lista de vehículos

Vehículo

Matricula
2222BBBB

Tipo
Tauliner

Capacidad
26T

Estado
activo

Javier Gonzalez

Administrador

Figura 33. Detalles del vehículo

Vehículo

Matricula

Tipo

Capacidad

Estado

Javier Gonzalez

Juan Pérez

Paco Martínez

Pepe Álvarez

saludos hola avatar

q w e r t y u i o p

a s d f g h j k l ñ

z x c v b n m

?123 , . ES • EN • PT

Figura 34. Nuevo vehículo

4.4.5 Tarea 5: Registro de jornada

El conductor puede registrar el inicio y fin de la jornada, así como su tipo. La aplicación calcula los tiempos acumulados y muestra avisos cuando se alcanza el límite permitido. En este tipo de dato se permite dejar la fecha y hora de fin sin registrar, pero la aplicación intentará que el conductor complete el registro que tiene pendiente antes de poder crear uno nuevo, esto se realiza comprobando si existe algún registro de ese conductor sin esos datos en la base de datos y en ese caso lo carga en la misma interfaz para poder completarlo. El administrador dispone del acceso a esta información ordenada por fecha para poder organizar la flota, además de poder crearlos o modificar la información que contienen para así poder ayudar a los conductores en el control de las actividades contribuyendo a la seguridad y al cumplimiento de la normativa vigente.



Figura 35. Picker fecha jornada

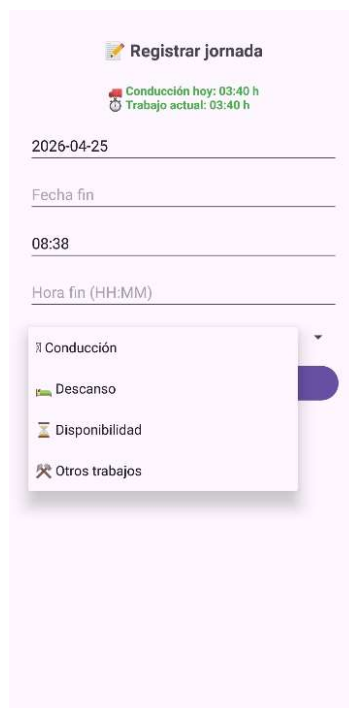


Figura 36. Tipo de jornada

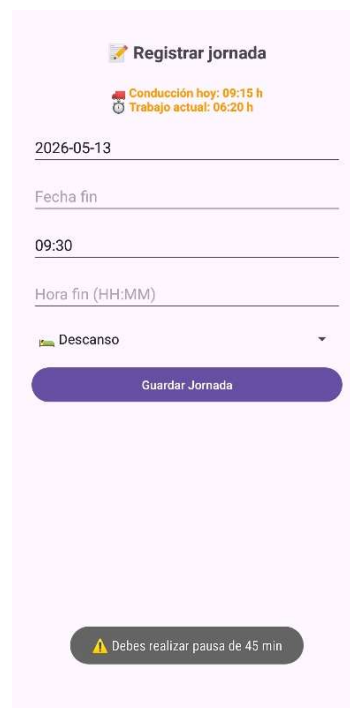


Figura 37. Aviso de jornada



Figura 38. Lista de registros

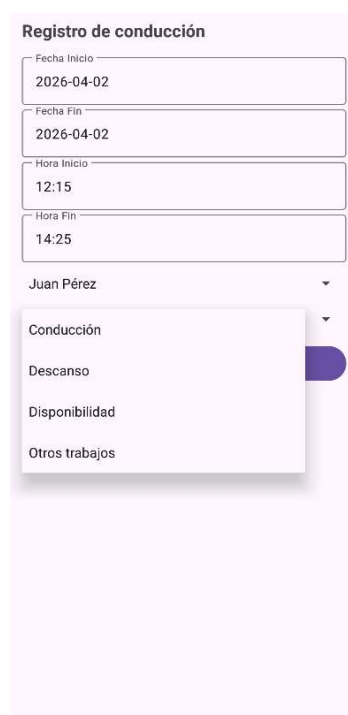


Figura 39. Crear registro



Figura 40. Picker fecha registro

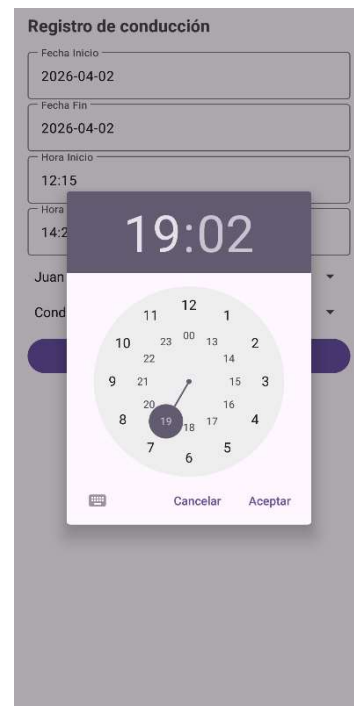


Figura 41. Picker hora registro

4.4.6 Tarea 6: Actualización de ubicación y compañeros cercanos

La aplicación permite compartir la ubicación del conductor con otros compañeros cercanos (que se encuentren a una distancia máxima aproximada de 25 km). El administrador, en cambio, puede visualizar en todo momento la ubicación toda la flota, hayan o no activado la opción de compartir ubicación, dado que para realizar su labor le es de mucha utilidad. Esta tarea facilita la coordinación de la flota y la ayuda entre los conductores. Para los conductores se añade la opción de contactar con el compañero que se encuentre en el rango de distancia mediante llamada telefónica, envío de mensaje sms o a través de la app de whatsapp.

4.4.7 Tarea 7: Aviso de avería

El conductor dispone de una opción específica para informar de incidencias, averías o eventos que crea necesario documentar. Al utilizar esta opción, aparece un indicador en el mapa en la ubicación en la que se encuentre en ese momento el conductor. El administrador también podrá crear avisos y será el encargado de modificarlos según se vayan desarrollando, así como de gestionarlos de la forma que considere oportuna.

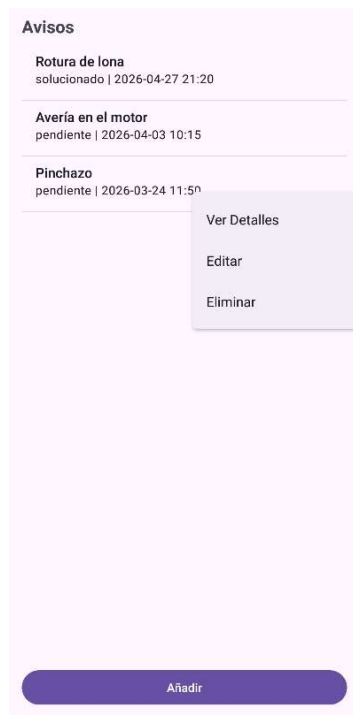


Figura 42. Lista de avisos

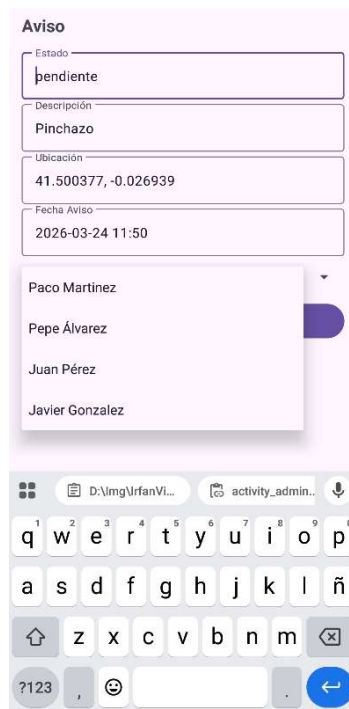
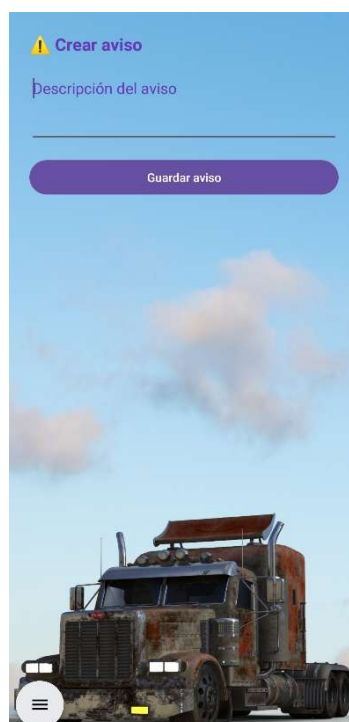


Figura 43. Editar avisos



Figura 44. Picker fecha aviso

El conductor para la creación de avisos únicamente ha de escribir la descripción ya que el sistema captura de manera automática la ubicación y la fecha simplificando esta tarea para el conductor.



4.4.8 Tarea 8: Generación de informes

El administrador podrá generar informes de todos los conductores a su cargo o de uno en concreto en cualquier momento además de enviarlo a través de email en formato escrito o como pdf. Esta actividad también programa un informe semanal con todos los conductores de cada administrador, el cual es generado y enviado el día elegido en el desplegable a media noche. Dicho informe es enviado desde la dirección de correo electrónico del usuario con rol master a cada usuario con rol admin a su respectivo email mediante contraseña de aplicación (no contraseña de usuario).

4.4.9 Tarea 9: Contactar

Es posible para un administrador el contactar con los conductores o con otros administradores a través de una ventana de diálogo en la que seleccionará en una lista desplegable con quien quiere comunicarse y mediante radio buttons marcará la forma de contactar. Esta comunicación puede ser realizada mediante llamada telefónica, envío de mensaje sms o a través de la aplicación de whatsapp.

4.4.10 Tarea 10: Gestión de administradores

Esta actividad solo se le permite realizarla al usuario con rol de master y se trata de crear, modificar o eliminar administradores, al igual que se hace con el resto de entidades que componen la aplicación. También se tiene en cuenta el tipo de dato a introducir para mostrar el teclado correspondiente y que cumpla patrones determinados.

4.5 Traducción de la aplicación

Debido a que en el sector del transporte existe una clara diversidad de nacionalidades y con el fin de hacerla más práctica se ha creado teniendo en cuenta la traducción a otros idiomas. Actualmente la app dispone del castellano como idioma principal y por defecto,

siendo esta traducida automáticamente según el idioma del dispositivo a inglés, portugués y rumano pudiendo ampliar el catálogo de idiomas en un futuro.

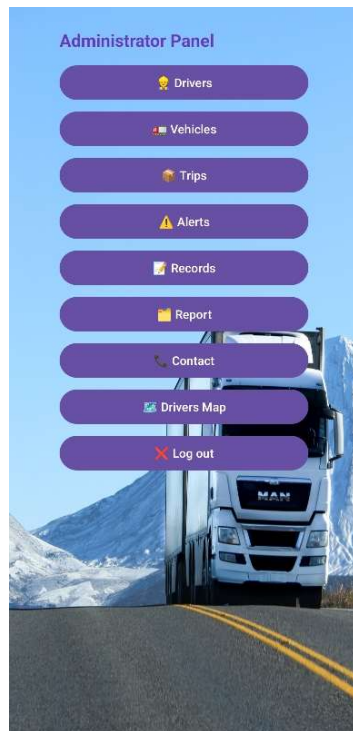


Figura 46. Inglés



Figura 47. Português



Figura 48. Rumano

4.6 Permisos requeridos

El correcto funcionamiento de la aplicación requiere la solicitud de varios permisos del sistema operativo Android, así como la configuración de diferentes componentes en el archivo AndroidManifest.xml y la inclusión de dependencias externas definidas en el archivo build.gradle.

En primer lugar, los permisos INTERNET y ACCESS_NETWORK_STATE permiten la comunicación de la aplicación con servicios externos, así como la comprobación del estado de la conexión a Internet antes de realizar operaciones en línea. Estos permisos son imprescindibles para el intercambio de información con la base de datos en la nube y otros servicios remotos.

Por otro lado, los permisos ACCESS_FINE_LOCATION y ACCESS_COARSE_LOCATION se utilizan para la obtención de la ubicación del usuario. El primero permite obtener una localización precisa mediante GPS, mientras que el segundo proporciona una ubicación aproximada utilizando redes móviles o Wi-Fi. Estos permisos son especialmente

importantes en el rol de conductor, ya que la aplicación requiere el seguimiento de su posición en tiempo real.

El permiso `FOREGROUND_SERVICE` permite la ejecución de servicios en primer plano. En este caso, se utiliza para mantener activo el servicio de localización continua, incluso cuando la aplicación no está en uso activo, garantizando así la recopilación constante de la ubicación del conductor durante los trayectos.

Además de los permisos, el `AndroidManifest.xml` define los distintos componentes de la aplicación, incluyendo múltiples actividades relacionadas con la gestión de usuarios, vehículos, viajes e informes. También se incluye un `FileProvider`, que permite el intercambio seguro de archivos con otras aplicaciones del sistema, un servicio en primer plano encargado del seguimiento de ubicación.

En cuanto a las dependencias definidas en el archivo `build.gradle`, la aplicación integra librerías externas necesarias para su funcionamiento. Entre ellas destacan `Google Play Services Location`, utilizada para la gestión de la localización del dispositivo, `OSMDroid` para la visualización de mapas, `Firebase Firestore` como base de datos en la nube, `Material Components` para la interfaz de usuario y `JavaMail` para el envío de correos electrónicos desde la aplicación.

En conjunto, todos estos elementos permiten que la aplicación funcione de forma completa, eficiente y adaptada a un entorno de gestión y seguimiento de conductores en tiempo real.

4.7 Gestión de Ubicaciones

En esta aplicación se utilizan dos enfoques diferentes para gestionar las ubicaciones: enviarlas a una aplicación externa para que las muestre y visualizarlas directamente en un mapa integrado dentro de la propia aplicación.

Para los conductores, se ha implementado una funcionalidad simple que permite mostrar la ubicación de los compañeros cercanos. Al seleccionar un conductor de la lista, su ubicación se transfiere automáticamente a la aplicación de mapas del dispositivo, lo que permite al usuario gestionarla desde allí.

En el caso de los administradores, se ha creado una vista que integra un mapa de `OpenStreetMap`, donde se muestran las ubicaciones de los conductores y los avisos. Esto proporciona al administrador una visión rápida y detallada de la situación en tiempo

real. Solo con pulsar sobre los iconos del mapa el administrador podrá ver el nombre del conductor o la descripción del aviso.

Las ubicaciones de los conductores son recopiladas por los dispositivos cada cinco minutos al igual que la actualización de las ubicaciones en el mapa del que disponen los administradores. Esta frecuencia se ha elegido cuidadosamente para evitar la sobrecarga de los dispositivos, ya que el monitoreo y recargas continuas de las ubicaciones puede consumir una cantidad significativa de batería. El objetivo ha sido desarrollar una aplicación eficiente en cuanto al consumo energético, para que los usuarios puedan utilizarla sin preocuparse por el agotamiento rápido de la batería.

4.8 Arquitectura

Para estructurar todas las funcionalidades de la aplicación se ha seguido el esquema habitual de actividades y vistas propio del desarrollo en Android. Además, se han implementado clases específicas para el acceso a los datos, junto con sus correspondientes clases POJO, así como diversas clases auxiliares encargadas de realizar tareas concretas e interactuar con el resto de componentes del sistema.

Cabe destacar que, en la parte destinada a la interacción del administrador con los datos más habituales, se ha diseñado una arquitectura diferente a la convencional. En este caso, se ha buscado fomentar la reutilización de vistas, con el objetivo de experimentar con diferentes enfoques de desarrollo. Esta decisión responde principalmente a la intención de ampliar conocimientos y adquirir mayor experiencia en programación y arquitectura de software. Asimismo, este planteamiento puede aportar valor añadido en caso de incluir la aplicación en un futuro portafolio, permitiendo demostrar el uso de otros conceptos en el diseño del código.

4.9 Diagramas del sistema

♦ Diagrama Entidad / Relación (E/R)

El diagrama Entidad/Relación (E/R) representa de forma gráfica la estructura de la base de datos del proyecto. Su objetivo principal es definir las entidades que intervienen en la aplicación, los atributos que describen cada entidad y las relaciones existentes entre ellas.

Este diagrama permite tener una visión clara y ordenada de cómo se almacenará la información y cómo se relacionan los distintos datos dentro del sistema, sirviendo como base para la posterior implementación de la base de datos.

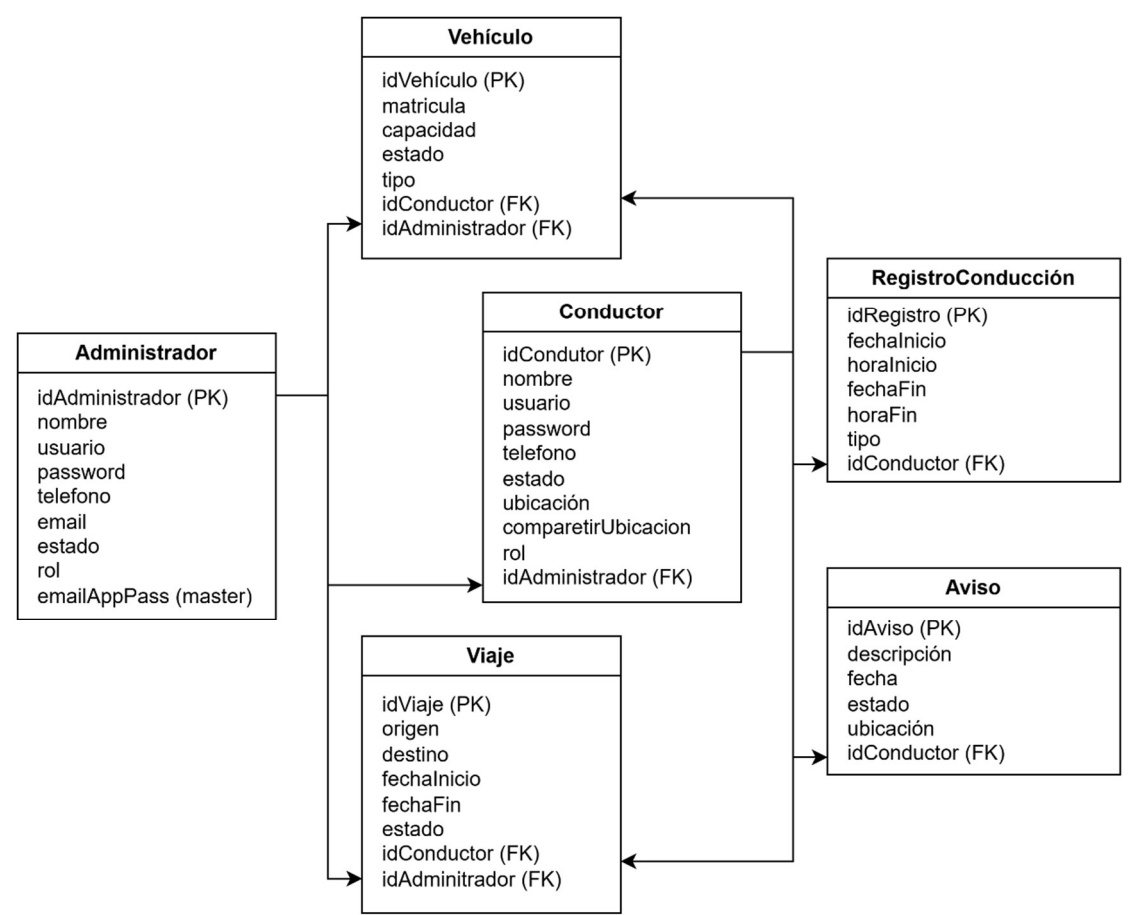


Figura 49. Diagrama Entidad/Relación (E/R)

◆ Diagrama de flujo

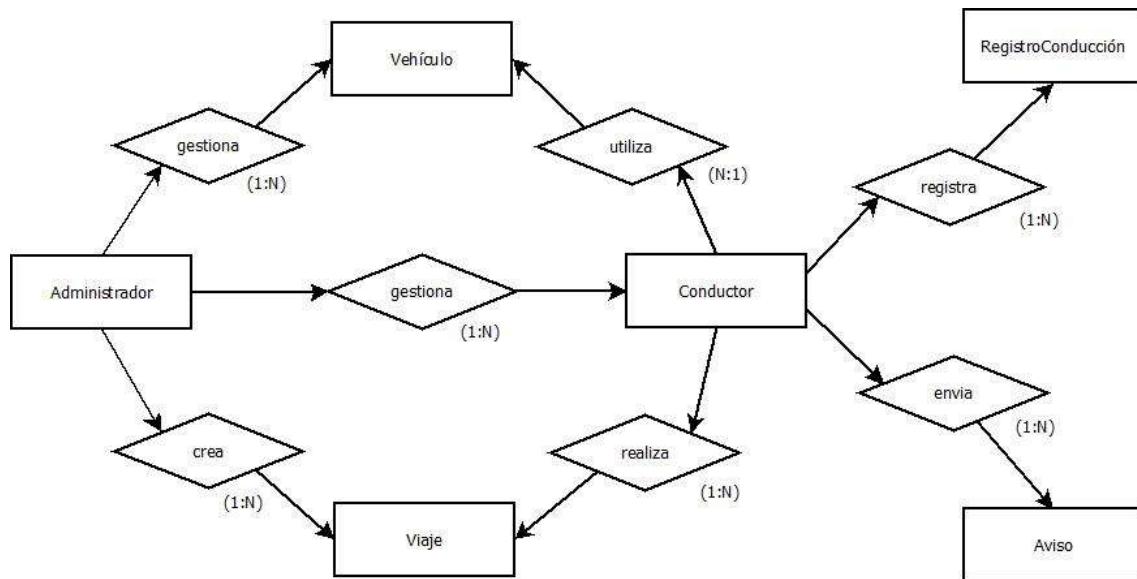


Figura 50. Diagrama de flujo

♦ Entidades principales del sistema

A continuación, se describen las entidades principales que forman parte del sistema y su función dentro de la aplicación.

▪ Administrador

La entidad Administrador (y master) representa a los gestores que acceden a la aplicación.

Atributos principales:

- idAdministrador (clave primaria)
- nombre
- usuario
- password
- telefono
- email
- estado
- rol
- emailAppPass (solo el master)

▪ Conductor

La entidad Conductor almacena la información relacionada con los conductores que realizan los viajes.

Atributos principales:

- idConductor (clave primaria)

nombre
usuario
password
teléfono
estado
ubicación
compartirUbicacion
rol
idAdministrador (clave foránea)

▪ Vehículo

La entidad Vehículo contiene los datos de los vehículos disponibles en la flota.

Atributos principales:

idVehiculo (clave primaria)
matrícula
capacidad
estado
tipo
idConductor (clave foránea)
idAdministrador (clave foránea)

▪ Viaje

La entidad Viaje representa cada uno de los desplazamientos realizados.

Atributos principales:

idViaje (clave primaria)
origen
destino
fechaInicio
fechaFin
estado
idConductor (clave foránea)
idAdministrador (clave foránea)

▪ RegistroConducción

La entidad RegistroConduccion almacena los registros de trabajo de los conductores.

Atributos principales:

idRegistro (clave primaria)
fechaInicio
horaInicio
fechaFin
horaFin

tipo
idConductor (clave foránea)

▪ Aviso

La entidad Aviso almacena los avisos de averías o problemas ocurridos durante un viaje.

Atributos principales:

idAviso (clave primaria)
descripción
fecha
estado
ubicación
idConductor (clave foránea)

♦ Relaciones entre entidades

Las entidades del sistema se relacionan entre sí de la siguiente forma:

- Un administrador puede gestionar o crear varios conductores, vehículos y viajes.
- Cada viaje, vehículo y conductor es gestionado o creado por un único administrador.
- Un conductor puede realizar varios viajes, pero cada viaje es realizado por un único conductor.
- Un vehículo puede estar asociado a varios conductores, pero cada conductor solo puede utilizar un vehículo.
- Un conductor puede enviar muchos avisos, pero cada aviso es enviado por un único conductor.
- Los registros son creados por un único conductor, pero cada conductor puede realizar muchos registros.

Estas relaciones permiten mantener la coherencia de los datos y facilitar la gestión de la información dentro de la aplicación.

♦ Justificación del diseño

El diseño del diagrama E/R se ha realizado buscando la simplicidad y claridad, evitando redundancias innecesarias y garantizando la integridad de los datos. La separación en entidades independientes facilita el mantenimiento y futuras ampliaciones del sistema.

4.10 Pruebas del sistema

En una aplicación de este tipo las pruebas son constantes durante el proceso de desarrollo del código basándose principalmente en la introducción, consulta, modificación y eliminación de los datos comprobando que todo sea correctamente almacenado en la base de datos. Para la realización de todas estas pruebas se ha monitorizado la base de datos para comprobar que esta se actualizaba correctamente en cada momento o que devolvía el dato adecuado solicitado por la aplicación.

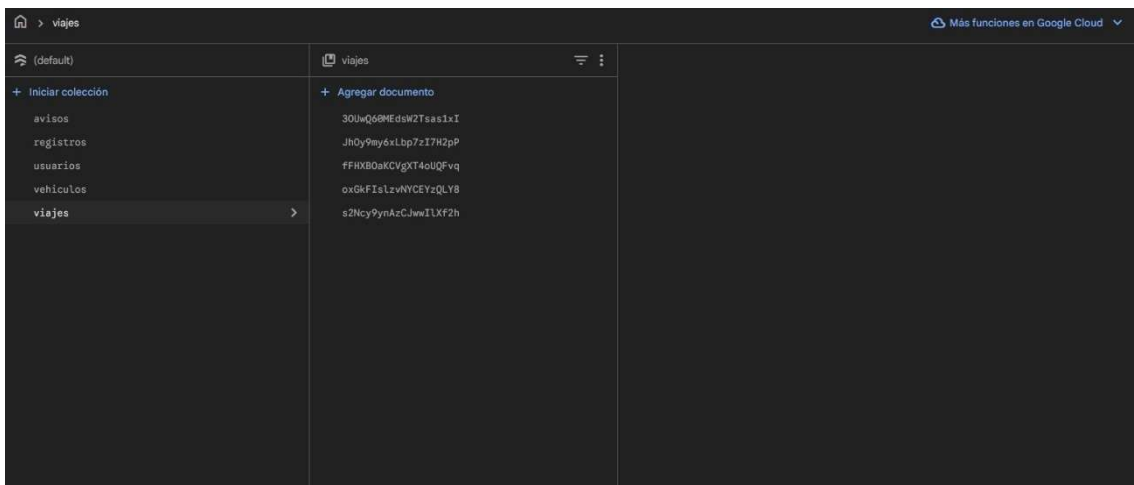


Figura 51. Estado de Firebase previo a la inserción de datos

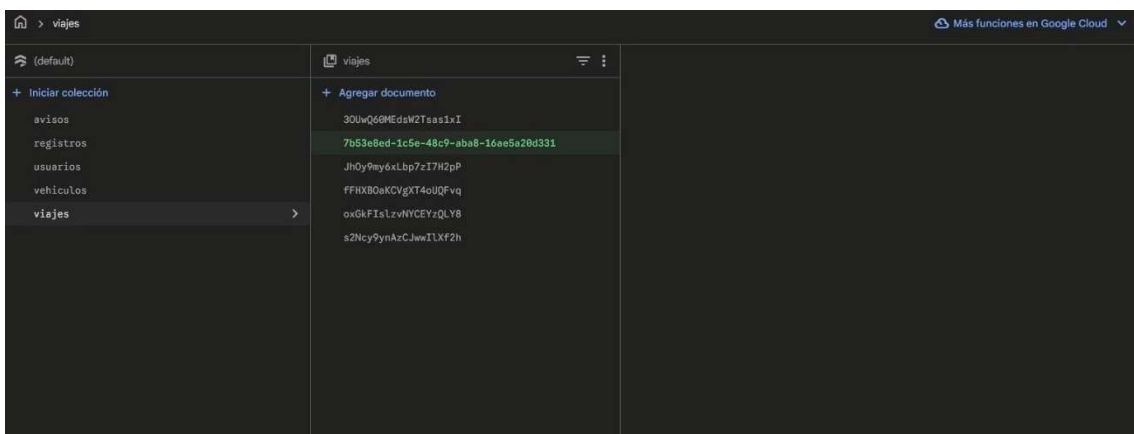


Figura 52. Inserción de datos desde la aplicación verificada en Firebase

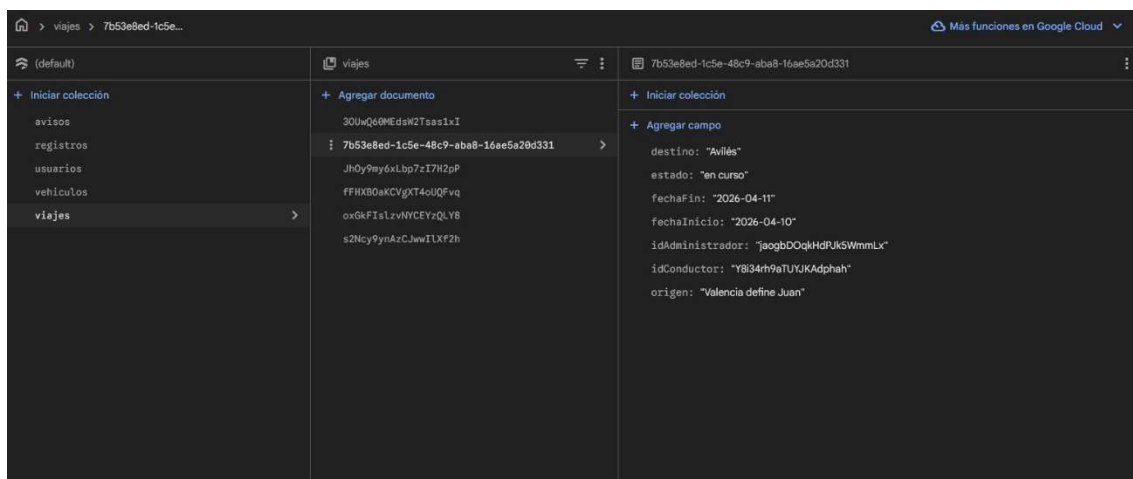


Figura 53. Comprobación del dato añadido correctamente

Las pruebas de funcionalidad también han sido importantes como las realizadas para comprobar la creación y envío de informes, la obtención y visualización de ubicaciones o el contacto entre usuarios. Estas pruebas han consistido en comprobar que la aplicación tenía una respuesta correcta a la tarea que el usuario le solicitaba dando paso a la aplicación correspondiente que se iba a encargar de ejecutar la tarea que nuestra app requería en cada momento.



Figura 54. Selección de aplicación

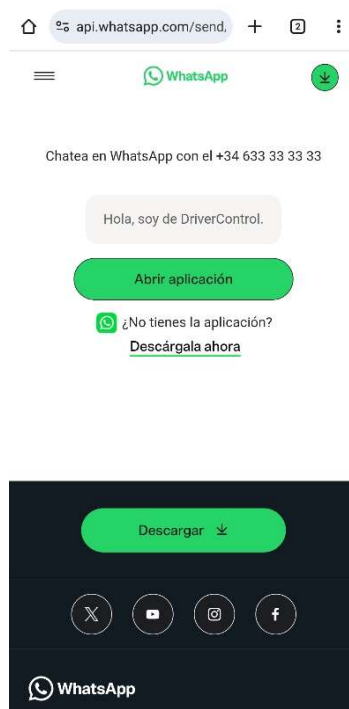


Figura 55. Envío de WhatsApp

A continuación, se incluyen los casos de prueba realizados para comprobar la funcionalidad de la aplicación.

ID	Funcionalidad	Entrada	Resultado esperado	Resultado obtenido
CP01	Login correcto	Usuario y contraseña válidos	Acceso al dashboard correspondiente	Correcto
CP02	Login incorrecto	Contraseña incorrecta	Mensaje de error	Correcto
CP03	Crear viaje	Datos completos	Viaje guardado	Correcto
CP04	Registro conducción	Datos jornada	Actualización de la jornada	Correcto
CP05	Compañeros cercanos	Ubicación determinada	Solo aparecen los conductores cercanos	Correcto
CP06	Envío informe email	Generar y enviar informe	Se abre app de correo	Correcto
CP07	Creación aviso	Conductor crea aviso	Se guarda con ubicación	Correcto
CP08	Actualización ubicación	Conductor con permisos	Actualización cada 5 min	Correcto
CP09	No compartir ubicación	Desactivar compartir ubicación	Compañero cercano no puede ver al conductor	Correcto
CP10	Traducción idiomas	Cambiar idioma dispositivo	Textos en inglés/portugués/rumano	Correcto
CP11	Gestión administradores (master)	Master crea nuevo admin	Admin añadido a Firebase	Correcto
CP12	Contactar WhatsApp	Seleccionar contacto y WhatsApp	Abre WhatsApp con contacto	Correcto

Tabla 7. Casos de prueba del sistema

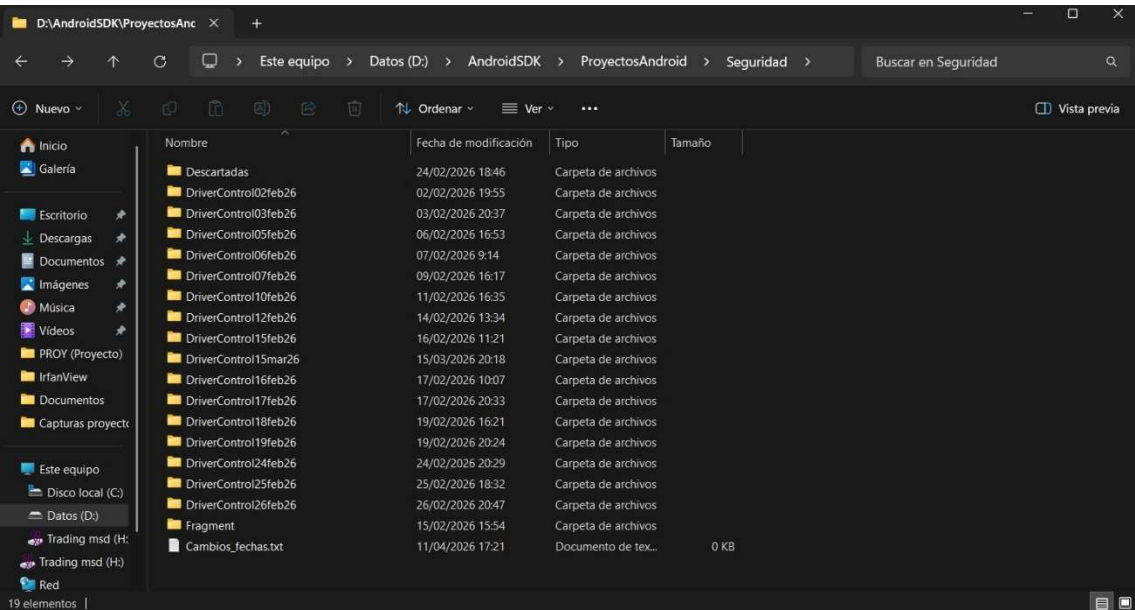
Resumen de las pruebas:

Total pruebas	Aprobadas	Fallidas	Corregidas
12	12	0	0

Tabla 8. Resumen de resultados

4.11 Control de cambios

En el desarrollo de esta aplicación se ha llevado un control de cambios a nivel local dado que es una aplicación desarrollada por un único programador en un único equipo. La metodología utilizada ha sido el almacenar frecuentemente los archivos de la aplicación en una carpeta, añadiéndole la fecha para poder tener un claro orden cronológico de las modificaciones creadas. A partir de tener una versión funcional se establecerá esta como la versión 1.0 (versión original) y se incluirá en las futuras mejoras el establecimiento de un control de versiones como Github o similar para constituir esta como la base de la que partan las posteriores versiones. El repositorio creado para tal fin es <https://github.com/saulsantosme/DriverControl> en el cual también se incluye el presente documento.



5. Conclusiones Finales

5.1 Conclusiones

A lo largo del desarrollo de este proyecto se ha diseñado una aplicación móvil para la gestión de una pequeña flota de transporte, cumpliendo con los objetivos planteados inicialmente. La aplicación permite al administrador supervisar viajes, conductores y vehículos, así como controlar los tiempos de conducción y descanso, mejorando la organización y la seguridad de la flota.

El proyecto ha permitido aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo, especialmente en el desarrollo de aplicaciones Android con Java, el uso de bases de datos y el diseño de interfaces orientadas al usuario. Asimismo, se ha trabajado la planificación del proyecto, la documentación y la resolución de problemas técnicos surgidos durante el desarrollo.

Aunque no se han implementado todas las funcionalidades previstas inicialmente, la aplicación desarrollada ofrece una base sólida y funcional que cumple con los requisitos principales. El resultado final demuestra la viabilidad del proyecto y la utilidad de la aplicación para empresas de transporte de pequeño tamaño, aportando una solución sencilla, eficiente y adaptable a futuras mejoras.

5.2 Grado de cumplimiento de los requisitos

A lo largo del desarrollo de este proyecto se ha diseñado una aplicación móvil para la gestión de una pequeña flota de transporte, intentando cumplir con los objetivos planteados inicialmente.

Cumplimiento de requisitos funcionales:

ID	Requisito	Cumplimiento
RF01	Inicio de sesión por rol	✓
RF02	Gestión de conductores (CRUD)	✓

RF03	Gestión de vehículos (CRUD)	✓
RF04	Gestión de viajes (CRUD)	✓
RF05	Mapa de ubicación de flota	✓
RF06	Generación de informes	✓
RF07	Contactar (llamada/SMS/WhatsApp)	✓
RF08	Consulta de viaje y vehículo por conductor	✓
RF09	Registro de tiempos	✓
RF10	Compartición opcional de ubicación	✓
RF11	Creación de avisos	✓
RF12	Contacto con administrador	✓
RF13	Gestión de administradores (master)	✓
RF14	Traducción a múltiples idiomas	⊙

Tabla 9. Cumplimiento de requisitos funcionales

Cumplimiento de requisitos no funcionales:

ID	Requisito	Cumplimiento
RNF01	Compatibilidad Android 8.0+	✓
RNF02	Conexión a Internet	✓
RNF03	Ubicación cada 5 minutos	✓
RNF04	Interfaz adaptada a vehículos	✓
RNF05	Reglas e índices de Firebase	✓
RNF06	Desarrollo en Java con Android Studio	✓

Tabla 10. Cumplimiento de requisitos no funcionales

Los requisitos tanto funcionales como no funcionales se han cumplido a excepción de la traducción que se ha cumplido de forma parcial. La interfaz principal está traducida, pero como se indica en el apartado de 'Propuestas de mejora', la traducción no es completa en todas las pantallas, elementos y mensajes del sistema. El proyecto ha

permitido aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo, especialmente en el desarrollo de aplicaciones Android con Java, el uso de bases de datos y el diseño de interfaces orientadas al usuario.

5.3 Propuestas de mejora y ampliaciones futuras

Como posibles mejoras futuras de la aplicación, se plantea la ampliación de funcionalidades orientadas a mejorar aún más la gestión de la flota y la experiencia de los usuarios. Entre estas mejoras se encuentra la incorporación de notificaciones en tiempo real mediante servicios push, lo que permitiría una comunicación más inmediata entre administradores y conductores. También se considera la mejora y ampliación de las traducciones, ya que las actuales no son completas, teniendo en cuenta los idiomas predominantes en el sector.

El sistema de informes automáticos parece tener un funcionamiento correcto, aunque habría que continuar realizando pruebas y modificarlo en caso necesario. Además, sería recomendable monitorizar el envío semanal, ya que se han realizado pruebas con periodos cortos de tiempo sin haber podido comprobar que en periodos largos funciona de igual manera. También se podrían ampliar estos informes, incluyendo estadísticas más detalladas, gráficos de rendimiento y análisis históricos.

La aplicación registra la jornada teniendo en cuenta el tipo de trabajo y realiza un cálculo de los tiempos, siendo un cálculo simple que debería mejorarse y desarrollarse más profundamente para que contemple las diferentes casuísticas que la normativa europea expone. Para esta funcionalidad, dada la complejidad de dicha normativa, sería más recomendable el uso de software o APIs externas.

Otra posible mejora sería la integración con sistemas externos, como plataformas de gestión empresarial, dispositivos GPS específicos instalados en los vehículos o con sistemas de obtención de datos directos del tacógrafo y descarga de tarjetas de conductor, pero esto podría requerir de software oficial de empresas como Siemens o Stoneridge al ser dispositivos precintados y sometidos a estrictas regulaciones.

Para el sector al que está destinada la aplicación sería interesante implementar un sistema de mantenimiento predictivo, registrando los kilómetros recorridos semanalmente. Esto permitiría al administrador programar los mantenimientos (cambio de aceite, revisión de neumáticos, ITV, etc.), mejorando la seguridad y reduciendo averías.

En cuanto a la seguridad, las reglas de Firebase utilizadas durante el desarrollo (modo prueba con fecha de expiración modificada) son suficientes para un entorno académico, pero en una versión profesional se deberían implementar reglas más estrictas para el control de los usuarios.

Además, se contempla la posibilidad de desarrollar una versión web complementaria para el administrador, facilitando el acceso desde cualquier dispositivo, así como la adaptación de la aplicación a otros sistemas operativos móviles. Estas mejoras permitirían convertir la aplicación en una solución más completa y escalable, adaptada al crecimiento de la empresa y a nuevas necesidades.

A partir de este momento en el cual ya se dispone de una versión básica pero funcional de la aplicación habría que integrar un sistema de versiones como puede ser Github o similar para establecer esta versión como la 1.0 (versión original) e ir ampliando y mejorando las diferentes funcionalidades y tareas que permite realizar esta aplicación para poco a poco obtener una versión más profesional y que todas esas mejoras queden registradas y documentadas.

6. Guías

6.1 Guía de uso

Primeramente, como es habitual en muchas aplicaciones, vamos a observar una pantalla de presentación mientras la aplicación realiza la carga de los datos del usuario, en caso de existir una sesión iniciada. Si no existiera sesión, lo próximo que veremos será la pantalla de login en la que introduciremos el usuario y la contraseña. Una vez hecho esto podremos acceder a las funcionalidades de la aplicación, existiendo también la posibilidad de editar nuestras credenciales de acceso para lo que nos aparecerá una ventana de diálogo en donde editarlas.

♦ Administrador (master)

Para el tratamiento de los datos relativos a los conductores, vehículos, viajes, avisos y registros el funcionamiento es el mismo. Según la opción escogida observaremos una lista con los principales datos existiendo un botón para añadir nuevos registros en la parte inferior. En caso de desear obtener más detalles, editar o eliminar algún elemento de la lista lo realizaremos desde un menú contextual que emerge de cada elemento. La

introducción de datos, editado y comprobación de detalles se realizan en otra vista que nos muestra todos los campos existiendo herramientas como pickers y spinner en los que seleccionar el dato a introducir para facilitar la tarea.

La pantalla de informe permite seleccionar desde un spinner si se desea un informe de todos los conductores o de un conductor concreto, los conductores que tenga asignado el administrador que está generando el informe. Una vez seleccionada la opción podremos pinchar en el botón de generar informe apareciendo en pantalla dicho informe. Con el informe ya generado podremos enviarlo a través de email como documento pdf adjunto o como texto del cuerpo del email para lo que se lanzará la aplicación correspondiente disponible en el dispositivo. Por último, en esta pantalla tenemos la posibilidad de modificar el día en que se genera el informe semanal de todos los conductores (sábado por defecto) el cual será enviado al email del administrador desde el email del usuario master.

La opción de contactar nos va a permitir lo que su propio nombre indica a través de una ventana de diálogo. En esta ventana vamos a poder escoger el contacto en un desplegable y seleccionar la forma de contactar mediante radio buttons, pudiendo elegir entre llamada, sms o WhatsApp. Una vez elegido, la funcionalidad pasará a realizarla la aplicación correspondiente instalada en el dispositivo.

El mapa de conductores se trata de una funcionalidad dentro de la aplicación que se implementa en una vista con OpenStreetMap en el que se colocan las diferentes ubicaciones de los conductores y de los avisos apareciendo con diferentes indicadores para una distinción rápida. Clicando en cualquiera de ellos nos aparecerá el nombre en el caso de los conductores o la descripción para los avisos. En la parte superior aparecen dos botones que como sus nombres indican uno realiza la labor de volver a la pantalla anterior (dashboard del administrador) y el otro actualiza las ubicaciones en pantalla del mapa (esta actualización también se ejecuta de forma automática cada cinco minutos).

La funcionalidad relativa a la gestión de los administradores solo está disponible para el usuario master. Esta pantalla contiene los campos de los datos del administrador en la parte superior incluyendo dos spinner con el rol y el estado. Tras introducir todos los datos podremos clicar en guardar (crear) o limpiar los campos en caso de habernos equivocado. Bajo estos botones aparece una lista con los administradores que actualmente tiene la aplicación en la que si clicamos en uno de ellos sus datos se rellenaran en la parte superior permitiendo consultarlos o modificarlos. En caso de querer eliminar un administrador se realizará mediante menú contextual en la lista.

Por último, tenemos la posibilidad de cerrar la sesión y volver a la pantalla de login.

◆ Conductor

Al logarnos y acceder al panel del conductor la aplicación nos solicitará los permisos para la ubicación, los cuales deberemos concederlos para poder tener la funcionalidad completa. Cabe destacar que para evitar sobrecargar el dispositivo con la monitorización constante de la ubicación se ha decidido obtenerla cada cinco minutos reduciendo notablemente el consumo de energía.

Una vez en el panel del conductor, se mostrarán sus datos y los del vehículo en la parte superior, dejando la parte central para ubicar los botones con los que interactuar. Las utilidades de viaje actual y vehículo simplemente muestran la información correspondiente para el conductor. En cuanto a crear aviso, únicamente nos va a pedir la descripción dado que el resto de datos los va a coger automáticamente del dispositivo (ubicación, fecha e id del conductor).

La pantalla para registrar la jornada muestra en la parte superior un mensaje con las horas conducidas haciendo el cálculo y teniendo en cuenta la normativa europea aplicable en este ámbito. Para registrar una nueva actividad va a solicitar al conductor las fechas y horas de inicio y fin, para lo cual van a desplegar sus correspondientes pickers. Además, dispone de un desplegable en el cual elegir en tipo de actividad que desea registrar. Estas actividades son las mismas que recoge la normativa de conducción para este tipo de trabajos (conducción, descanso, disponibilidad y otros trabajos). Por último, con el botón guardar jornada se actualiza la base de datos. Para este tipo de datos se permite dejar la fecha y hora de fin sin cumplimentar siendo la aplicación la encargada de recargar el registro que tengamos pendiente de completar cuando volvamos a esta pantalla.

En el caso de que el conductor quiera contactar con el administrador dispone de la opción con este mismo nombre en la que le muestra la información del administrador y bajo ella un botón de contactar. Este botón lanza una ventana de diálogo en la que se muestran las diferentes formas de contactar (llamada, sms o WhatsApp) en la que con tan solo pulsar la opción deseada se redirigirá la acción a la aplicación adecuada instalada en el dispositivo.

Si un conductor en algún momento desea saber si tiene algún compañero cerca este puede utilizar el botón de compañeros cercanos el cual lanza una vista con una lista en la que aparecen los contactos de los compañeros que se encuentran en un rango de distancia (25km aproximadamente). Encima de la lista existe un switch que permite al conductor compartir u ocultar su ubicación con el resto de conductores en caso de que no desee compartir esta información con los compañeros (el administrador continúa teniendo acceso a ella). La lista de los conductores cercanos tiene una doble funcionalidad como es conocer la ubicación de estos y además poder contactar con ellos

directamente. Ambas funcionalidades se desarrollan de manera similar seleccionando el conductor en la lista y pulsando el botón correspondiente de la parte inferior. Para contactar además existen tres radio buttons con las opciones disponibles (llamada, sms y WhatsApp) de las cuales seleccionaremos la deseada antes de pulsar el botón de contactar que lanzará la aplicación correspondiente. El botón ver en maps lanzará la aplicación maps del dispositivo con la ubicación del conductor seleccionado en la lista.

El botón cerrar sesión hace lo que su propio nombre indica devolviendo a la aplicación a la pantalla de login.

6.2 Guía de instalación

La aplicación se ha desarrollado para dispositivos Android y está diseñada para gestionar los datos de cada cliente de forma independiente, garantizando la privacidad mediante el uso de Firebase como plataforma de backend. Firebase proporciona servicios esenciales para la aplicación, como reglas de acceso, índices para el acceso a los datos, almacenamiento de estos datos en tiempo real mediante Cloud Firestore y la capacidad de enviar correos electrónicos de manera segura utilizando contraseñas de aplicación almacenadas en los documentos. Gracias a estas funcionalidades, la aplicación puede operar sin almacenar información sensible directamente en el código y con total seguridad para cada cliente.

Dado que cada cliente requiere datos privados e independientes, se ha adoptado un modelo en el que cada cliente dispone de un proyecto propio, único e independiente de Firebase. Esto significa que los datos de un cliente no pueden ser accedidos desde la aplicación de otro cliente, garantizando la confidencialidad y el aislamiento de la información. La instalación de la aplicación puede llevarse a cabo mediante dos enfoques principales. El primero, denominado instalación manual, consiste en configurar la conexión a Firebase a través de la consola web, descargando y colocando los archivos de configuración necesarios y asegurando que la estructura de datos se crea correctamente. El segundo, denominado instalación automática, utiliza el asistente integrado de Firebase en Android Studio para realizar la configuración de manera rápida y sin necesidad de modificar manualmente archivos o descargar configuraciones adicionales. Ambos métodos son válidos, y la elección depende de la preferencia del desarrollador y del nivel de control deseado sobre la configuración del proyecto.

Antes de iniciar el proceso de instalación, se requiere contar con Android Studio instalado y correctamente configurado en el equipo de desarrollo. Además, es necesario disponer de una cuenta de Google con acceso a Firebase y tener a disposición un dispositivo Android o un emulador donde ejecutar la aplicación. También se requieren

conocimientos básicos sobre la navegación en Firebase Console y en Android Studio, para poder crear proyectos, usuarios y colecciones en Firestore.

El primer paso en la instalación consiste en crear un proyecto Firebase para cada cliente. Para ello, se accede a Firebase Console y se selecciona la opción para crear un nuevo proyecto. A cada proyecto se le asigna un nombre identificativo (por ejemplo, ClienteNombre-App), que idealmente debe reflejar el nombre del cliente, de manera que se mantenga un vínculo claro entre el proyecto Firebase y la instalación de la aplicación correspondiente. Una vez creado el proyecto, la aplicación requiere un usuario master para la administración de la app y el envío seguro de correos electrónicos mediante contraseña de aplicación. Este usuario permite la gestión de datos críticos sin necesidad de exponer credenciales en el código de la app.

Posteriormente, se debe crear un usuario master dentro del proyecto Firebase. Este usuario tiene privilegios administrativos y es necesario para el correcto funcionamiento de la aplicación. Su principal función es la gestión de los datos de los administradores, así como el envío de correos electrónicos periódicos mediante una contraseña de aplicación y su correo electrónico, además de disponer de todas las funcionalidades del resto de administradores. Esta contraseña especial evita que la contraseña real del correo se exponga en el código de la aplicación o se almacene en la base de datos. La creación de este usuario no es imprescindible para el funcionamiento de la aplicación, solo para las funcionalidades concretas destinadas a dicho usuario. El usuario master se configura con un "Email", una "Contraseña", un "Rol" definido como "master" y un identificador de "Usuario". Además, se debe incluir un valor para la contraseña de aplicación "emailAppPass" (se obtiene en la cuenta de correo electrónico) que se utilizará dentro de la app para enviar correos.

Un ejemplo de usuario master (datos mínimos) es el siguiente:

Campo	Valor
email	drivercontrol.master@gmail.es
password	master
rol	master
usuario	master
emailAppPass	zdukcmoproahqvkt

Tabla 11. Datos mínimos para el usuario master

Es recomendable complementar el usuario master con datos similares a los del resto de administradores (nombre, password, teléfono, estado) para disponer de la totalidad de las funcionalidades, pero no es imprescindible para el funcionamiento de la aplicación.

Este usuario debe crearse de manera manual en Firebase, dentro de la colección usuarios, y además deben configurarse las reglas necesarias en Firestore, como la fecha de expiración que Firebase incluye por defecto en el modo de prueba, la cual bloquea el acceso a la base de datos tras 30 días. Para evitarlo, se debe modificar esta fecha a un valor posterior o establecerla como 'true' para que no expire. También debemos añadir los siguientes índices para que los datos listados aparezcan ordenados de manera adecuada y facilitar la utilización:

ID de la colección	Campos indexados
avisos	↑ idConductor - ↑ fecha
registros	↑ idConductor - ↑ fechaInicio - ↑ horaInicio
registros	↓ fechaInicio - ↓ horaInicio
usuarios	↑ compartirUbicacion - ↑ rol - ↑ nombre
usuarios	↑ rol - ↑ nombre
usuarios	↑ idAdministrador - ↑ rol - ↑ nombre

Tabla 12. Índices necesarios para Firebase

Estos índices también pueden establecerse desde el log de Android Estudio, si después de haber establecido la conexión de la app con la base de datos lanzamos la app aparecerá un log de error, este nos llevará a los índices de firebase y nos lo definirá de forma automática solo teniendo que pinchar en crear. Habría que hacerlo tantas veces como índices tengamos ya que se pararía la ejecución en cada índice no creado.

Este procedimiento se repite de forma independiente para cada cliente, asegurando que todos los datos queden aislados y no se mezclen entre clientes.

Una vez creado el proyecto y el usuario master, se procede a conectar la aplicación con Firebase. En la forma manual, esta conexión se establece descargando el archivo de configuración google-services.json desde Firebase Console y colocándolo en la carpeta correspondiente del proyecto. Además, se debe verificar que el archivo de Gradle de la aplicación contiene el plugin de Firebase, lo que permite que Android Studio reconozca los servicios de Firebase y sincronice la configuración con el proyecto. Tras colocar el archivo y configurar Gradle, se realiza la sincronización del proyecto para asegurar que todos los componentes de Firebase estén disponibles dentro de la aplicación. Ejemplo del build.gradle(app):

```
plugins {  
    alias(libs.plugins.android.application) apply false  
    alias(libs.plugins.google.gms.google.services) apply false  
}
```

Este método proporciona más control sobre la base de datos y permite personalizar la configuración para cada cliente, aunque requiere un manejo manual de los archivos de configuración y puede ocasionar errores si se gestionan múltiples clientes.

El segundo método consiste en utilizar el asistente de Firebase integrado en Android Studio. Este asistente permite conectar la aplicación con Firebase sin necesidad de descargar archivos ni modificar Gradle manualmente. Para ello, se abre Android Studio y en el menú de herramientas se accede a la opción de Firebase. Se selecciona el servicio que se desea utilizar, como Cloud Firestore o Cloud Storage for Firebase, y se pulsa el botón para conectar con Firebase. Tras iniciar sesión con la cuenta de Google, se selecciona el proyecto correspondiente al cliente. Android Studio realiza automáticamente la descarga del archivo google-services.json, configura el plugin de Firebase en Gradle y añade las dependencias necesarias para que la aplicación pueda interactuar con los servicios seleccionados. Este enfoque es mucho más rápido y sencillo, reduciendo la posibilidad de errores de configuración.

Una vez establecida la conexión con Firebase mediante cualquiera de los métodos, se procede a generar el archivo de instalación de la aplicación, el APK. En Android Studio se accede al menú de construcción (build) y se selecciona la opción para generar APK. Tras finalizar el proceso, el APK se encuentra en la carpeta de salida del proyecto y puede transferirse a los dispositivos de los clientes mediante correo electrónico, almacenamiento en la nube o cualquier otro medio seguro. Para instalarlo, es necesario permitir la instalación desde orígenes desconocidos en el dispositivo Android. Cada APK generado estará vinculado al proyecto Firebase del cliente correspondiente, por lo que para cada cliente se debe generar un APK independiente asegurando que la aplicación se conecte únicamente a la base de datos y los datos del cliente adecuado.

Durante todo el proceso de configuración, es importante tener en cuenta las recomendaciones de seguridad. En particular, no se deben incluir contraseñas reales en el código de la aplicación; siempre se debe utilizar la contraseña de aplicación proporcionada por el proveedor de correo electrónico. Esto asegura que la información de cada cliente permanezca protegida y que la aplicación cumpla con los requisitos de privacidad y seguridad establecidos.

En conclusión, la instalación y configuración de la aplicación requiere de varios pasos coordinados que van desde la creación del proyecto en Firebase y del usuario maestro, hasta la conexión de la aplicación mediante la forma manual o el asistente de Android

Studio y la generación del APK para su instalación en los dispositivos de los clientes. Cada método de conexión tiene sus ventajas y limitaciones, pero ambos permiten que la aplicación funcione correctamente y que los datos de cada cliente permanezcan privados y aislados. La creación del usuario master y la inicialización de documentos en Firestore son imprescindibles para que la aplicación opere completa desde el primer inicio, y el cumplimiento de las recomendaciones de seguridad garantiza que la gestión de los datos sea segura y confiable.

6.3 Mapa de navegación del sitio

♦ Introducción

El mapa de navegación del sitio representa de forma esquemática la estructura de la aplicación y el recorrido que puede realizar el usuario entre las distintas pantallas. Su función principal es facilitar la comprensión de cómo se organiza la aplicación y cómo se accede a cada una de sus funcionalidades.

Este mapa permite identificar las diferentes secciones del sistema y la relación entre ellas, sirviendo como guía tanto para el desarrollo como para el uso de la aplicación.

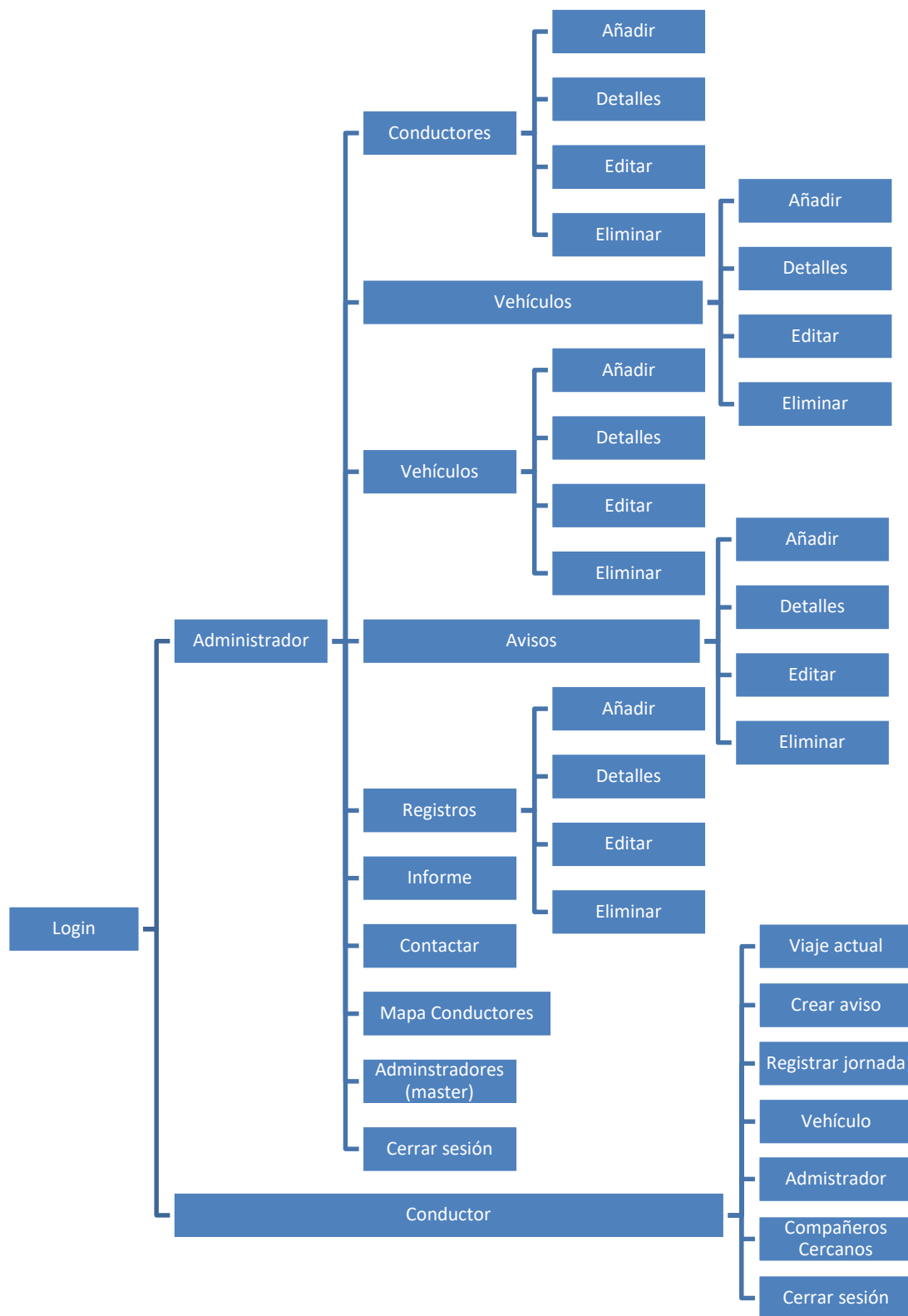


Figura 57. Mapa de navegación

♦ Estructura general de la navegación

La aplicación se organiza en torno a una pantalla inicial desde la cual el usuario puede acceder al resto de funcionalidades según su rol. La navegación se ha diseñado para que sea clara, intuitiva y sencilla, evitando recorridos innecesarios.

La estructura general es la siguiente:

- ▶ Pantalla de inicio de sesión.
- ▶ Menú principal según el tipo de usuario con sus funcionalidades específicas.
- ▶ Diferentes pantallas según cada funcionalidad.
- ▶ Pantallas de gestión y consulta.
- ▶ Cierre de sesión.

♦ Descripción de las pantallas principales

▪ Pantalla de carga

Al iniciar la app se lanza una pantalla que se muestra durante un par de segundos cuya función es dar tiempo a la aplicación a cargar los datos necesarios y poder determinar si existe algún usuario logado e iniciar su sesión.

▪ Pantalla de inicio de sesión

Es la primera pantalla que se muestra al acceder a la aplicación. Permite al usuario introducir sus credenciales para acceder al sistema. En función del rol del usuario, se mostrará un menú adaptado a sus permisos. También permite modificar dichas credenciales mediante una ventana de diálogo.

▪ Menú principal

Desde el menú principal el usuario puede acceder a todas las secciones disponibles de la aplicación. Este menú actúa como punto central de navegación.

▪ Gestión de conductores

Esta sección permite visualizar, añadir, modificar y eliminar conductores. El acceso a esta y a las siguientes opciones está limitado a usuarios con permisos de administrador o master.

▪ Gestión de vehículos

Desde esta pantalla se puede consultar la flota de vehículos, así como añadir nuevos vehículos o modificar los existentes.

- Gestión de viajes

Aquí nos permite consultar los viajes, acceder a sus detalles, crear nuevos y modificarlos según vayan transcurriendo para mantener actualizada la información.

- Gestión de avisos

En esta sección se muestran las incidencias registradas durante los viajes. Permite añadir nuevas incidencias y consultar su estado.

- Registros

Permite al conductor registrar sus tiempos de trabajo y descanso. Por su parte, el administrador también puede gestionarlos al igual que hace con el resto de la información para la que está destinada la app.

- Informe

Esta pantalla permite a los administradores generar y enviar informes (de todos los conductores o de uno en concreto) y enviarlo a través de email además de seleccionar el día en que desean que se les envíe el informe semanal programado.

- Contactar

Para esta utilidad realmente no se utiliza una vista completa, sino que se realiza a través de una ventana de diálogo en la que elegir el contacto y la forma en que se desea contactar. Por su parte los conductores si disponen de una vista en la que les aparece la información del administrador y desde la cual pueden contactar con él.

- Gestión de administradores

Esta vista es exclusiva del usuario master y le permite consultar la información de los administradores, así como crearlos o editar la información de estos.

- Mapa, ubicación

La vista que proporciona la información sobre ubicación es diferente en según si el usuario sea administrador o conductor. Los administradores disponen de una vista con un mapa en que aparecen las ubicaciones de los conductores y de los avisos.

Los conductores simplemente verán un selector donde elegir si quieren que sus compañeros puedan ver su ubicación y les aparecerá una lista con los compañeros que se encuentren cerca permitiendo contactar o ver la ubicación de este en la aplicación maps del dispositivo.

- Cierre de sesión

Permite al usuario salir de la aplicación de forma segura, finalizando la sesión activa.

♦ Mapa de navegación

El mapa de navegación se ha diseñado con el objetivo de ofrecer una experiencia de usuario sencilla y ordenada. Todas las funcionalidades principales están accesibles desde el menú principal, lo que facilita el uso de la aplicación y reduce el tiempo necesario para encontrar cada opción. En general el flujo se realiza mediante botones, dejando las tareas menos realizadas en manos de menús contextuales (ver detalles, editar y eliminar) o de ventanas de diálogo para las acciones que requieren de menos interacción (contactar).

Para facilitar la utilización de la aplicación se han utilizados pickers en los campos que precisan de la introducción de horas o fechas, además de spinners en los campos con valores determinados (ej. tipos de registros) o donde es posible seleccionar datos obtenidos desde la base de datos (ej. conductores), esto además garantiza una mayor consistencia en los datos.

7. Referencias

En este apartado se recogen las principales fuentes de información y documentación que se han utilizado como apoyo para el desarrollo del proyecto. Estas referencias han servido como guía para el diseño de la aplicación, la toma de decisiones técnicas y la correcta documentación del sistema.

A continuación, se enumeran las principales referencias empleadas durante el desarrollo del proyecto:

- Documentación oficial de Android Developers.
- Apuntes y material proporcionado en el ciclo formativo.
- Documentación sobre Java y programación orientada a objetos.
- Recursos sobre diseño de interfaces móviles y experiencia de usuario.
- Manuales y guías sobre bases de datos relacionales y no relacionales.
- Documentación técnica sobre Firebase y servicios backend.
- Recursos online y tutoriales especializados en desarrollo Android.
- Blogs de profesores como abrilcode y codeandcoke.

Las referencias han sido utilizadas como apoyo para:

- Comprender el funcionamiento de las herramientas de desarrollo.
- Implementar correctamente las funcionalidades de la aplicación.
- Diseñar una arquitectura adecuada y escalable.

- Resolver problemas técnicos durante el desarrollo.
- Documentar correctamente el proyecto.

A continuación, se presentan algunas de las principales referencias bibliográficas utilizadas durante el desarrollo del proyecto.

Android Developers. (s.f.). *Android developer guides*. Google.

<https://developer.android.com/guide>

Android Developers. (s.f.). *Activities and app components*. Google.

<https://developer.android.com/guide/components/activities>

Android Developers. (s.f.). *Intents and intent filters*. Google.

<https://developer.android.com/guide/components/intents-filters>

Android Developers. (s.f.). *Request app permissions*. Google.

<https://developer.android.com/training/permissions/requesting>

Android Developers. (s.f.). *PendingIntent reference*. Google.

<https://developer.android.com/reference/android/app/PendingIntent>

Android Developers. (s.f.). *WorkManager overview*. Google.

<https://developer.android.com/topic/libraries/architecture/workmanager>

Faci, S. (2025). *Tutoriales y recursos educativos sobre desarrollo Android*.

<https://codeandcoke.com>

Firebase. (s.f.). *Firebase for Android: Documentation*. Google.

<https://firebase.google.com/docs/android/setup>

Firebase. (s.f.). *Cloud Firestore documentation*. Google.

<https://firebase.google.com/docs/firestore>

Google Developers. (s.f.). *Android Studio overview*. Google.

<https://developer.android.com/studio>

Google Developers. (s.f.). *Google Maps Platform documentation*. Google.

<https://developers.google.com/maps/documentation>

Google Developers. (2024). *Android Developers Documentation*. Google.

<https://developer.android.com>

Material Design. (s.f.). *Material Design Guidelines*. Google.

<https://m3.material.io/>

OpenStreetMap Foundation. (s.f.). *OpenStreetMap project*.

<https://www.openstreetmap.org>

OSMDroid Project. (s.f.). OSMDroid: *Android library for OpenStreetMap*.

<https://github.com/osmdroid/osmdroid>

Oracle. Java documentation. (s.f.). *Oracle Corporation*.

<https://docs.oracle.com/en/java/>

Stack Overflow. (s.f.). *Stack Overflow developer community*.

<https://stackoverflow.com>

Valdeón, F. (2026). *Tutoriales y recursos educativos sobre programación*.

<https://www.abrilcode.com>

Además, se ha utilizado el material didáctico del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma (DAM).

8. Preguntas frecuentes

En este apartado se recogen algunas de las preguntas más habituales que pueden surgir en relación con el funcionamiento y el uso de la aplicación de gestión de flotas, junto con sus correspondientes respuestas.

- ¿Quién puede utilizar la aplicación?

La aplicación está pensada para dos tipos de usuarios: el administrador de la flota y los conductores. Cada perfil dispone de permisos y funcionalidades diferentes según su rol.

- ¿Es necesario disponer de conexión a Internet para utilizar la aplicación?

Sí, la aplicación requiere conexión a Internet para el acceso a la base de datos, la actualización de ubicaciones, el acceso a los datos y la generación de informes. En caso de no tener conexión esta no se actualizará y utilizará los datos almacenados en el caché.

- ¿La aplicación permite controlar los tiempos de conducción y descanso?

Sí, los conductores pueden registrar sus tiempos de conducción y descanso, y el sistema calcula automáticamente los tiempos permitidos, indicando cuando se alcanzan los límites establecidos.

- ¿La ubicación de los conductores es visible para todos los usuarios?

La ubicación de los conductores es siempre visible para el administrador. La visualización entre conductores es opcional y puede activarse o desactivarse para preservar la privacidad.

- ¿Se pueden generar informes automáticamente?

Sí, el administrador puede generar informes de manera manual o programar su generación automática, recibiendo por correo electrónico.

- ¿Qué ocurre si un conductor tiene una avería o necesita ayuda?

El conductor puede crear un aviso desde la aplicación y esta incluirá su ubicación exacta.

- ¿Es imprescindible el usuario master?

Este usuario hace posible las tareas de gestionar los administradores o del envío de informes programados pero el resto de funcionalidades son independientes a su existencia por lo que no es imprescindible su creación, pero en su defecto habría que crear un usuario administrador.

9. Gráficos y figuras

Figura 1. Diagrama de Gantt	11
Figura 2. Estructura de la base de datos en Firebase	17
Figura 3. Reglas de seguridad configuradas en Firebase	18
Figura 4. Índices configurados en Firebase	19
Figura 5. Pantalla de inicio	20
Figura 6. Pantalla de login	21
Figura 7. Panel del administrador	22
Figura 8. Panel del master	22
Figura 9. Lista de viajes	23
Figura 10. Editar viaje	23
Figura 11. Picker fecha de viaje	23
Figura 12. Confirmación eliminar	23
Figura 13. Selección conductor informe	24
Figura 14. Informe conductor	24
Figura 15. Informe en email	25

Figura 16. Selección día informe	25
Figura 17. Selección de contacto	26
Figura 18. Ventana de contacto	26
Figura 19. Contacto administrador	26
Figura 20. Conductor en el mapa	27
Figura 21. Aviso en el mapa	27
Figura 22. Gestión de administradores	28
Figura 23. Panel del conductor	29
Figura 24. Datos vehículo	29
Figura 25. Compañeros cercanos	30
Figura 26. Ubicación en maps	30
Figura 27. Editar credenciales	31
Figura 28. Contraseña no válida	31
Figura 29. Viaje actual	32
Figura 30. Lista de conductores	33
Figura 31. Editar conductor	33
Figura 32. Lista de vehículos	34
Figura 33. Detalles del vehículo	34
Figura 34. Nuevo vehículo	34
Figura 35. Picker fecha jornada	35
Figura 36. Tipo de jornada	35
Figura 37. Aviso de jornada	35
Figura 38. Lista de registros	35
Figura 39. Crear registro	35

Figura 40. Picker fecha registro _____	36
Figura 41. Picker hora registro _____	36
Figura 42. Lista de avisos _____	37
Figura 43. Editar avisos _____	37
Figura 44. Picker fecha aviso _____	37
Figura 45. Describir aviso _____	37
Figura 46. Inglés _____	39
Figura 47. Portugués _____	39
Figura 48. Rumano _____	39
Figura 49. Diagrama Entidad/Relación (E/R) _____	42
Figura 50. Diagrama de flujo _____	43
Figura 51. Estado de Firebase previo a la inserción de datos _____	46
Figura 52. Inserción de datos desde la aplicación verificada en Firebase _____	46
Figura 53. Comprobación del dato añadido correctamente _____	47
Figura 54. Selección de aplicación _____	47
Figura 55. Envío de WhatsApp _____	47
Figura 56. Control de cambios _____	49
Figura 57. Mapa de navegación _____	61
Tabla 1. Requisitos funcionales del sistema _____	6
Tabla 2. Requisitos no funcionales del sistema _____	7
Tabla 3. Riesgos laborales durante el desarrollo del proyecto _____	8
Tabla 4. Análisis DAFO – Factores internos _____	15
Tabla 5. Análisis DAFO – Factores externos _____	16

Tabla 6. Justificación del modelo de la base de datos por cliente _____	19
Tabla 7. Casos de prueba del sistema _____	48
Tabla 8. Resumen de resultados _____	49
Tabla 9. Cumplimiento de requisitos funcionales _____	50
Tabla 10. Cumplimiento de requisitos no funcionales _____	51
Tabla 11. Datos mínimos para el usuario master _____	57
Tabla 12. Índices necesarios para Firebase _____	58

11. Anexo

Arquitectura, funcionamiento, localización e informes

Sección A – Arquitectura general del sistema

♦ Introducción

En este anexo se explica cómo está construida la aplicación DriverControl por dentro, es decir, su arquitectura software y las decisiones técnicas que fueron tomando durante el desarrollo. El objetivo es mostrar que el sistema no solo funciona, sino que está pensado para que se pueda ampliar fácilmente, se pueda mantener sin mucho esfuerzo y se aprovechen al máximo los componentes creados.

♦ Arquitectura por capas

La aplicación sigue una arquitectura dividida en tres capas bien diferenciadas. La primera es la capa de presentación, que se encarga de todo lo que el usuario ve y toca: las pantallas, los botones, los formularios y la navegación. Lo importante aquí es que esta capa solo gestiona la interfaz, pero no contiene lógica de negocio ni acceso a la base de datos. Todo eso lo delega en las capas inferiores.

La segunda capa es la lógica dinámica, gracias a ella, la aplicación no necesita tener una pantalla distinta para cada tipo de dato (conductores, vehículos, avisos, viajes, etc.), sino que se reutilizan las mismas pantallas para todo. Esto se consigue mediante un enumerado llamado TipoLista y una clase auxiliar que usa reflexión de Java. A continuación, se muestra un fragmento del enumerado donde se define la configuración para los conductores:

```
CONDUCTORES(  
    "Conductor",  
    new String[]{"Usuario", "Nombre", "Teléfono", "Contraseña",  
"Estado", "Ubicación", "Id Administrador"},  
    new String[]{"usuario", "nombre", "telefono", "password",  
"estado", "ubicacion", "idAdministrador"},  
    Conductor.class  
) ,
```

Como se observa, cada entrada del enumerado contiene el título de la pantalla, las etiquetas de los campos, los nombres de las propiedades de la clase y la clase asociada.

La tercera capa es la de persistencia de datos, implementada en una clase llamada FirestoreDB. Esta clase centraliza todas las operaciones contra Firebase Firestore: crear

registros, actualizarlos, eliminarlos, hacer consultas personalizadas, generar informes y también gestionar la ubicación de los conductores. A continuación, se muestra el método genérico que permite crear cualquier tipo de entidad:

```
// Crea una entidad en Firestore (sirve para cualquier tipo de objeto)
public <T> void crearEntidad(T entidad, Runnable ok, Runnable error) {
    String coleccion = getColeccion(entidad);
    String id = getId(entidad);

    // Si no se puede determinar colección o id → error
    if (coleccion == null || id == null) {
        error.run();
        return;
    }

    // Guarda el objeto completo en Firestore
    db.collection(coleccion)
        .document(id)
        .set(entidad)
        .addOnSuccessListener(a -> ok.run())
        .addOnFailureListener(e -> error.run());
}
```

Este método es totalmente genérico: recibe cualquier objeto, deduce automáticamente en qué colección debe guardarse (el método `getColeccion` realiza un `instanceof`) y cuál es su identificador para ejecutar la operación. Al tener todo centralizado, la interfaz gráfica no accede directamente a la base de datos, lo que reduce el acoplamiento entre componentes y facilita los cambios futuros.

♦ Gestión de usuarios y roles

El sistema distingue entre varios tipos de usuarios mediante un control de roles. Por un lado, está el administrador, que tiene acceso a casi todas las funciones y puede supervisar a los conductores. Por otro lado, está el conductor, que solo puede ver y hacer cosas relacionadas con su propio trabajo. Además, existe un usuario máster, que es como un superadministrador: puede gestionar a los administradores y es quien tiene la contraseña especial (SMTP) que se utiliza para el envío automático de correos. Dependiendo del rol que tenga el usuario que se acaba de autenticar, la aplicación muestra u oculta ciertos botones y opciones del menú. En el siguiente fragmento de `ListActivity` se puede ver cómo se comprueba el rol antes de permitir el acceso:

```
// Comprobamos el rol del usuario guardado en la sesión
SharedPreferences prefs = getSharedPreferences("sesion",
MODE_PRIVATE);
String rol = prefs.getString("rol", null);

// Si no es admin o master, lo mandamos al login
if (rol == null || (!rol.equals("admin") && !rol.equals("master"))) {
    startActivity(new Intent(this, LoginActivity.class));
    finish();
}
```



```
        return;  
    }
```

♦ Seguridad

En cuanto a la seguridad, se han aplicado varias medidas. La más importante es que el acceso a las funciones depende del rol del usuario. Además, las credenciales de correo para enviar informes no están escritas en el código, sino que se guardan en la base de datos, dentro del documento del usuario master:

```
// Obtenemos el email y la contraseña de aplicación del master  
String masterEmail = master.getDocuments().get(0).getString("email");  
String masterPass =  
master.getDocuments().get(0).getString("emailAppPass");
```

También se controla que las consultas a Firestore estén restringidas para que un usuario no pueda acceder a datos que no le corresponden, como se observa en el método `obtenerConductoresPorAdmin`, que filtra los conductores por el identificador del administrador logueado.

Sección B – Gestión dinámica y funcionamiento interno

♦ El enumerado `TipoLista`

La clave de la reutilización de pantallas está en un enumerado llamado `TipoLista`. Cada valor de este enumerado contiene la configuración necesaria para trabajar con una entidad del sistema: el título que debe llevar la pantalla, qué campos del formulario hay que mostrar y a qué clase del modelo pertenece esa entidad. Gracias a esto, las actividades no necesitan saber con qué tipo de dato están trabajando, porque toda la información la obtienen de forma dinámica a partir del `TipoLista` que reciben a través de un extra del Intent.

♦ `ListActivity` como listado universal

La actividad que muestra listas de datos se llama `ListActivity` y funciona como un visor genérico. Cuando el usuario entra en un módulo, esta actividad recibe a través de un Intent el `TipoLista` correspondiente. Con esa información configura su título, pide los datos a `FirestoreDB` y los muestra en pantalla usando un adaptador reutilizable (`ListAdapter`). Desde ahí se pueden hacer operaciones como crear, editar o eliminar elementos mediante menús contextuales. De esta forma no hace falta crear una actividad distinta para cada entidad del sistema.

♦ ItemDetailActivity como formulario dinámico

Para crear, consultar o modificar datos se usa una sola actividad llamada ItemDetailActivity, que puede trabajar en tres modos diferentes: creación, edición o solo consulta. Lo interesante es que el formulario no está fijo. Al abrir la pantalla, todos los campos comienzan ocultos y luego se van mostrando solo aquellos que están definidos en la configuración del TipoLista correspondiente. El siguiente fragmento muestra cómo se ocultan inicialmente todos los campos y luego se habilitan solo los necesarios:

```
// Ocultamos todo al inicio
for (int i = 0; i < layouts.length; i++) {
    layouts[i].setVisibility(View.GONE);
    campos[i].setVisibility(View.GONE);
}
spinner1.setVisibility(View.GONE);
spinner2.setVisibility(View.GONE);

// Spinner 1 que selecciona el conductor
for (int i = 0; i < hints.length; i++) {
    String prop = propiedades[i];
    if ("idConductor".equals(prop)) {
        spinner1.setVisibility(View.VISIBLE);

        // configurar spinner de conductores...

    } else {
        layouts[i].setVisibility(View.VISIBLE);
        campos[i].setVisibility(View.VISIBLE);
        layouts[i].setHint(hints[i]);
    }
}
```

Las etiquetas de los campos y los valores que se muestran también se asignan automáticamente. Además, este formulario integra spinners que se cargan desde Firestore, selectores de fecha y hora, y validación previa antes de guardar.

♦ ReflectionUtils y la reflexión de Java

Para que todo esto sea posible sin tener que escribir código específico para cada clase, se ha creado una clase llamada ReflectionUtils que aprovecha el mecanismo de reflexión de Java. La reflexión permite que el programa pueda leer y modificar propiedades de un objeto en tiempo de ejecución sin conocer de antemano la clase concreta. Con esta técnica, el sistema puede invocar los getters y setters de cualquier propiedad simplemente sabiendo su nombre. El siguiente método obtiene los valores de un objeto a partir del nombre de la propiedad o atributo:

```
// Obtenemos el valor de una propiedad
public static String getProperty(Object objeto, String propiedad) {
    try {
        String getPropiedad = "get" + capitalize(propiedad);
        Method getter = objeto.getClass().getMethod(getPropiedad);
        Object valor = getter.invoke(objeto);
        return valor != null ? valor.toString() : "";
    } catch (Exception e) {
        e.printStackTrace();
        return "";
    }
}
```

Así, cuando el usuario rellena un formulario, los valores se asignan automáticamente al objeto correspondiente sin necesidad de programar campo por campo.

♦ FirestoreDB y el acceso a datos

La clase FirestoreDB sigue un patrón Singleton, lo que significa que solo existe una instancia de esta clase durante toda la ejecución de la aplicación. Esto reduce el consumo de recursos y centraliza el acceso a Firestore. Su implementación es la siguiente:

```
public class FirestoreDB {

    // Instancia de la clase
    private static FirestoreDB instance;
    private final FirebaseFirestore db;

    private FirestoreDB() {
        db = FirebaseFirestore.getInstance();
    }

    // Devuelve siempre la misma instancia de FirestoreDB
    public static FirestoreDB getInstance() {
        if (instance == null) instance = new FirestoreDB();
        return instance;
    }
}
```

Los métodos principales son genéricos, es decir, funcionan con cualquier tipo de entidad. Por ejemplo, el método crearEntidad recibe un objeto y automáticamente deduce en qué colección debe guardarlo y cuál es su identificador. Para las actualizaciones parciales, los objetos se convierten a Map, de modo que solo se modifican los campos que han cambiado sin tener que sobrescribir el documento entero. Como Firestore es asíncrono, todas las operaciones usan callbacks para no bloquear la interfaz gráfica mientras se espera la respuesta del servidor.

♦ Flujo general de la aplicación

El funcionamiento de la aplicación es el siguiente. El usuario selecciona un módulo desde el menú. Se abre ListActivity que recibe el TipoLista correspondiente y pide los datos a FirestoreDB. Cuando el usuario pulsa sobre un elemento, se abre ItemDetailActivity, que construye el formulario dinámicamente según la configuración y, con ayuda de ReflectionUtils, carga los datos del objeto seleccionado. Si el usuario modifica algo y lo guarda, ReflectionUtils se encarga de asignar los nuevos valores al objeto y FirestoreDB lo persiste en la nube.

♦ Ventajas de este diseño

La principal ventaja es la reutilización de código. Añadir una nueva funcionalidad al sistema solo requiere crear la clase del modelo, añadir una entrada en el enumerado TipoLista y definir la colección correspondiente en FirestoreDB. No sería necesario modificar las actividades existentes haciendo que el mantenimiento sea más sencillo y que la aplicación esté preparada para crecer sin necesidad de reescribir partes importantes.

Sección C – Sistema de localización y mapas

♦ Introducción

El módulo de localización permite seguir a los conductores en tiempo real y ver su posición en un mapa, junto con los avisos que han ido creando. Para ello se combinan un servicio en segundo plano, Firestore como base de datos y la librería OpenStreetMap para los mapas.

♦ Captura de ubicación en segundo plano

La obtención de la ubicación del dispositivo se hace mediante un Foreground Service llamado LocationForegroundService. Este servicio sigue ejecutándose, aunque la aplicación no esté en primer plano, de modo que la ubicación se sigue actualizando incluso si el conductor deja la app en segundo plano. El servicio utiliza el proveedor de localización fusionado de Android, que combina GPS, redes WiFi y antenas de telefonía para obtener la ubicación de manera eficiente y sin gastar demasiada batería. A continuación, se muestra la configuración de la obtención programada cada cinco minutos:

```
// Configuramos cada cuánto tiempo se pedirá la ubicación (cada 5 minutos)
```

```

LocationRequest locationRequest = new
LocationRequest.Builder(Priority.PRIORITY_BALANCED_POWER_ACCURACY, 5 *
60 * 1000).build();

// Definimos qué hacer cuando se recibe una nueva ubicación
locationCallback = new LocationCallback() {
    @Override
    public void onLocationResult(@NonNull LocationResult
locationResult) {
        Location location = locationResult.getLastLocation();
        if (location != null) {
            // Enviamos la ubicación a Firestore
            enviarUbicacionAFirestore(location);
        }
    }
}

```

Las actualizaciones también se producen cada cinco minutos, lo que da un equilibrio entre tener una ubicación actualizada y no consumir recursos en exceso.

♦ Almacenamiento de la ubicación

Cuando se obtiene una nueva ubicación, se convierte en un texto sencillo con la latitud y la longitud separadas por una coma. Después se asocia al conductor que tiene la sesión iniciada y se actualiza el campo correspondiente en Firestore mediante la clase FirestoreDB. De este modo, la base de datos siempre tiene la última ubicación conocida de cada conductor.

```

// Método que envía la ubicación a la base de datos
private void enviarUbicacionAFirestore(Location location) {

    // Obtenemos el id del conductor guardado en SharedPreferences
    SharedPreferences prefs = getSharedPreferences("sesion",
MODE_PRIVATE);
    String idConductor = prefs.getString("idConductor", null);
    if (idConductor == null) return;

    // Convertimos la latitud y longitud en un String
    String ubicacion = location.getLatitude() + "," +
location.getLongitude();

    // Actualizamos la ubicación del conductor en Firestore
    FirestoreDB.getInstance().actualizarUbicacionConductor(idConductor,
ubicacion);
}

```

♦ Visualización en el mapa

La pantalla donde se ven los mapas se llama MapaConductoresActivity y utiliza OpenStreetMap. Al abrirla, el sistema carga todos los conductores desde Firestore, filtra aquellos que tienen una ubicación válida y crea un marcador para cada uno. Además,

también busca los avisos que están en estado pendiente y que tienen una ubicación asignada, y crea marcadores adicionales con un aspecto diferente. El siguiente fragmento muestra cómo se crea un marcador para un conductor:

```
String[] partes = c.getUbicacion().split(",");
double lat = Double.parseDouble(partes[0].trim());
double lon = Double.parseDouble(partes[1].trim());
GeoPoint point = new GeoPoint(lat, lon);

Marker marker = new Marker(mapView);
marker.setPosition(point);
marker.setTitle(c.getNombre());
marker.setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM);
mapView.getOverlays().add(marker);

if (centro[0] == null) centro[0] = point; // Primer punto para centrar
mapa
```

Todo este proceso se repite periódicamente mediante un Handler y su método `postDelayed`, de forma que el mapa se va actualizando automáticamente para reflejar los cambios en tiempo real.

◆ Conductores cercanos

La clase `UbicacionActivity` se encarga de calcular qué conductores están cerca del que ha iniciado sesión. Para simplificar el cálculo y que no sea demasiado pesado, no se usan fórmulas geodésicas complejas, sino una aproximación basada en rangos de latitud y longitud:

```
// Rango aproximado de cercanía (~25 km)
double rangoLat = 0.23;
double rangoLon = 0.3;

if (Math.abs(latConductorLogueado - lat) <= rangoLat &&
    Math.abs(lonConductorLogueado - lon) <= rangoLon) {
    listaCercanos.add(c);
}
```

Los conductores que se encuentren dentro de ese rango aparecen en una lista, y desde ahí se puede interactuar con ellos: llamar, enviar un SMS, abrir WhatsApp o ver su ubicación en Google Maps.

◆ Privacidad y control de compartición

El sistema incluye un interruptor que permite al conductor decidir si quiere compartir su ubicación con sus compañeros. Cuando el interruptor cambia de estado, se actualiza un campo en Firestore:

```
switchCompartirUbicacion.setOnCheckedChangeListener((buttonView,
isChecked) -> {
    db.actualizarConductorCompartirUbicacion(idConductorLogueado,
isChecked,
    () -> Toast.makeText(this, "Estado de compartir ubicación
actualizado", Toast.LENGTH_SHORT).show(),
    () -> Toast.makeText(this, "Error al actualizar el estado",
Toast.LENGTH_SHORT).show()
    );
});
```

Si el conductor tiene desactivada la compartición, los demás conductores no podrán ver su posición, aunque el administrador siempre puede verla independientemente de esta opción. Es importante aclarar que el sistema sigue obteniendo la ubicación del dispositivo, aunque la compartición esté desactivada, lo único que cambia es si esa ubicación se muestra o no al resto de conductores.

♦ Optimización del sistema

Para no gastar demasiada batería ni consumir muchos datos, se han tomado varias decisiones. Se usa ubicación fusionada en lugar del GPS en modo exclusivo. Las actualizaciones no son continuas sino cada varios minutos. El servicio en segundo plano solo está activo cuando es necesario. La carga del mapa se hace de forma periódica pero no constante. Y los datos que se envían a la nube son muy ligeros. Con todo esto se consigue que la funcionalidad en tiempo real no agote la batería del dispositivo.

Sección D – Generación de informes y envío de correos

♦ Generación de informes

Una de las funcionalidades importantes del proyecto es la generación de informes. El usuario administrador puede seleccionar un conductor concreto o todos los conductores que tenga asignados, y el sistema genera automáticamente un documento con la información que saca de la base de datos. Internamente, el proceso consiste en recuperar los datos, procesarlos con clases auxiliares, construir el contenido del informe y mostrarlo en pantalla. Para que este proceso sea reutilizable y no haya que programar un informe distinto para cada tipo de dato, se vuelven a usar las utilidades de reflexión de las que se habla arriba, que permiten obtener las propiedades de los objetos de forma dinámica. A continuación, se presenta un ejemplo de la estructura básica de los informes, no es completo sino una representación simplificada ya que la clase realiza múltiples comprobaciones y cálculos:

```
sb.append("📄 INFORME SEMANAL\n");
sb.append("Conductor: ").append(nombreConductor).append("\n");
```

```
sb.append("• Avisos: ").append(avisos.size()).append("\n");
sb.append("• Registros: ").append(registros.size()).append("\n");
```

Los informes realmente son tres diferentes, el de todos los conductores es como una estadística, el de cada conductor contiene lo referente a ese conductor y el automático es de todos los conductores enlazando los individuales.

♦ Envío de correos

Una vez generado el informe, el sistema ofrece la posibilidad de enviarlo por correo electrónico. Se puede enviar como texto plano o convertirlo a PDF realizando el envío mediante las aplicaciones de correo que el usuario tenga instaladas en su dispositivo. A continuación, se muestra un resumen del cómo se crea el pdf, la versión que utiliza la aplicación es algo más extensa:

```
PdfDocument pdf = new PdfDocument();

// Tamaño de página A4 en puntos (595x842)
PdfDocument.PageInfo pageInfo = new PdfDocument.PageInfo.Builder(595,
842, 1).create();
PdfDocument.Page page = pdf.startPage(pageInfo);
Canvas canvas = page.getCanvas();

int x = 40;           // margen horizontal
int y = 50;           // margen vertical
int maxWidth = 515;   // ancho máximo de texto

// Dibujamos cada línea del texto en el PDF
for (String linea : texto.split("\n")) {
    canvas.drawText(linea, x, y, paint);
    y += 20; // espacio entre líneas
}
```

♦ Programación automática de informes

Además del envío manual, el sistema permite programar informes periódicos para que se generen y envíen automáticamente sin que el usuario tenga que interactuar. Esto se implementa con un Worker de Android, que es un componente que ejecuta tareas en segundo plano cuando se cumplen ciertas condiciones. El siguiente fragmento de código muestra la programación de dicho worker:

```
// Calculamos la primera ejecución
Calendar cal = Calendar.getInstance();
cal.setFirstDayOfWeek(Calendar.SUNDAY); // La semana empieza por
domingo para evitar problemas
cal.set(Calendar.DAY_OF_WEEK, diaSemana);
cal.set(Calendar.HOUR_OF_DAY, HORA_INFORME);
cal.set(Calendar.MINUTE, MINUTO_INFORME);
cal.set(Calendar.SECOND, 0);
```



```

// Si la fecha ya pasó, se programa para la próxima semana
if (cal.getTimeInMillis() <= System.currentTimeMillis()) {
    cal.add(Calendar.DAY_OF_YEAR, 7);
}

// Cálculo del retraso inicial hasta la primera ejecución
long delay = cal.getTimeInMillis() - System.currentTimeMillis();

// Datos para el worker
Data inputData = new Data.Builder().putString("ID_ADMIN",
idAdmin).build();

// Trabajo periódico cada 7 días
PeriodicWorkRequest workRequest = new PeriodicWorkRequest.Builder(
InformeWorker.class, 7, TimeUnit.DAYS).setInitialDelay(delay,
TimeUnit.MILLISECONDS).setInputData(inputData).build();

// Reemplaza el trabajo si existiera otro con el mismo id
WorkManager.getInstance(this).enqueueUniquePeriodicWork(
"informe_semanal_" + idAdmin,
ExistingPeriodicWorkPolicy.REPLACE,
workRequest
);

```

El administrador puede seleccionar un día de la semana, y a partir de ese momento el sistema se encarga de todo solo: genera el informe, prepara el correo y lo envía al destinatario. Para que el envío automático funcione sin intervención del usuario, se utilizan credenciales SMTP. No se utiliza la contraseña normal del correo, sino una contraseña de aplicación que se guarda en Firestore mejorando así la seguridad. En el código que se añade a continuación se observa la configuración del servidor para el envío automático:

```

// Configuramos el servidor SMTP de Gmail
Properties props = new Properties();
props.put("mail.smtp.host", "smtp.gmail.com"); // Servidor de
Gmail
props.put("mail.smtp.port", "587"); // Puerto TLS
props.put("mail.smtp.auth", "true"); // Autenticación
requerida
props.put("mail.smtp.starttls.enable", "true"); // Usar TLS
props.put("mail.smtp.ssl.trust", "smtp.gmail.com"); // Confiar en el
certificado

// Creamos la sesión con autenticación
Session session = Session.getInstance(props, new
javax.mail.Authenticator() {
    @Override
    protected PasswordAuthentication getPasswordAuthentication() {
        return new PasswordAuthentication(user, pass);
    }
});

// Creamos el mensaje del email
MimeMessage msg = new MimeMessage(session);
msg.setFrom(new InternetAddress(user));
msg.setRecipient(Message.RecipientType.TO, new
InternetAddress(destino));

```

```
msg.setSubject("Informe Semanal DriverControl");  
msg.setText(texto);
```

♦ Limpieza automática de datos antiguos

Para evitar que la base de datos crezca indefinidamente, el sistema incorpora un mecanismo de limpieza automática. Los registros, avisos y viajes que superen los cuatro meses de antigüedad se eliminan automáticamente. Como estos tipos de datos contienen fechas se comparan con la actual eliminando los establecidos. Para esta tarea es importante el cálculo de la antigüedad por meses en el cambio de año, eso lo realiza el siguiente fragmento de código:

```
Calendar cal = Calendar.getInstance();  
int mesTmp = cal.get(Calendar.MONTH) - mesesAntigüedad;  
if (mesTmp < 0) mesTmp += 12;  
final int mesLimite = mesTmp;
```

De esta forma la base de datos se mantiene en un tamaño razonable y no se acumula información obsoleta.

♦ Conclusión del anexo

A lo largo de este anexo se han presentado los fragmentos de código más relevantes del proyecto DriverControl, intentando mostrar que el sistema no solo funciona, sino que está construido siguiendo buenas prácticas de software: arquitectura por capas, patrones de diseño (Singleton, Builder, etc.), reutilización de componentes mediante reflexión y enumerados, gestión eficiente de localizaciones, y generación automática de informes con envío por correo programado. La inclusión de estos fragmentos de código fuente permite detallar cómo se han implementado las decisiones técnicas descritas.